Unidad 4:
Lenguajes Tipo 1 y Tipo 0

Jerarquía de Chomsky (1956) – Lenguaje Tipo 1

- Son más expresivos que los lenguajes independientes de contexto (Tipo 2) y menos que los sin restricciones (Tipo 0).
- Se caracterizan porque el contexto influye en cómo se interpretan las reglas, lo que permite describir estructuras más complejas que las que puede manejar un lenguaje independiente de contexto.
- Se utilizan en situaciones donde la posición o el entorno de un símbolo importa, por ejemplo, en algunos aspectos de lenguajes naturales y en ciertas construcciones de lenguajes de programación.
- Son importantes porque muestran que no todo puede resolverse ignorando el contexto, y marcan un nivel intermedio en la jerarquía de Chomsky.

Reglas Gramaticales – Lenguaje Tipo 1

- La característica de este tipo de gramática es que:
  - el lado izquierdo de las producciones contiene al menos un símbolo no terminal
  - la cadena sustituta, que está en el lado derecho de las producciones nunca será vacía, por lo tanto es una secuencia de símbolos terminales y/o no terminales

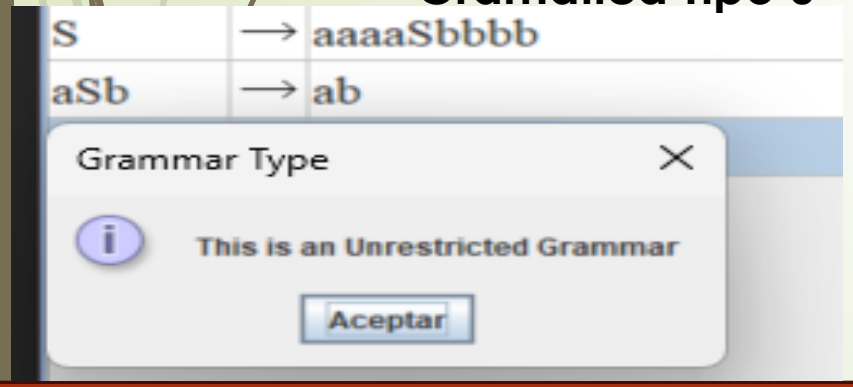
- La restricción es que:
  - la cantidad de símbolos que forman la cadena del lado izquierdo es menor o a lo sumo igual que la cantidad de símbolos de la cadena del lado derecho.

UTN, FRRO – Apunte de Cátedra – Capítulo 2: Gramáticas
https://www.frro.utn.edu.ar/repositorio/catedras/sistemas/2_anio/sintaxis/SySL_cap5_2015_Gram%C3%A1ticas.pdf

Ejemplo Gramáticas Lenguajes Tipo 1 – Tipo 0

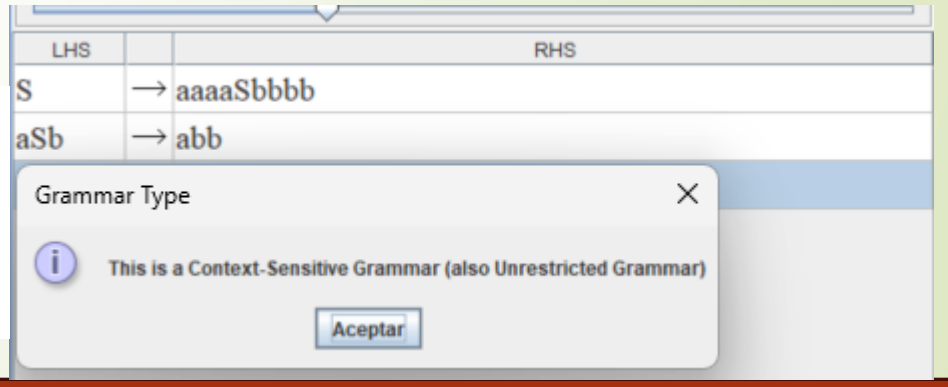
- **Tipo 1 (Gramáticas Sensibles al Contexto):** Sus producciones $\alpha \rightarrow \beta$ imponen la condición de **no decrecimiento** $|\alpha| \leq |\beta|$ lo que garantiza que la longitud de las cadenas nunca decrezca a lo largo de una derivación.
- **Tipo 0 (Gramáticas sin Restricciones):** No imponen ninguna limitación a las producciones $\alpha \rightarrow \beta$ permitiendo reemplazar o borrar símbolos deliberadamente.

Gramática Tipo 0



Son gramáticas distintas pero que diferencia a una de otra ¿Que diferencia un lenguaje de otro?

Gramática Tipo 1



Forma Normal Kuroda (FNK)

- Para los lenguajes de Tipo 1 principalmente, se utiliza en la FNK que permite las siguientes reglas:

A → BC

A → B

AB → CD

A → a

Tipo	Forma normal
Tipo 3	Regular / lineal
Tipo 2	Chomsky / Greibach
Tipo 1	**Kuroda (restringida)**
Tipo 0	**Kuroda (general)**

No escribiremos gramáticas Tipo 1 o Tipo 0



Sige-Yuki Kuroda
Nacido en Tokio, Japón
1 de Agosto1934-2009
Lic. en Matemáticas
Máster en Matemáticas
Doctorado en Lingüística

SOLO CONOCER GENERALIDADES

Ejercicio 1 – Gramática ¿Esta en FNK?

Recordatorio

$A \rightarrow BC$
$A \rightarrow B$
$AB \rightarrow CD$
$A \rightarrow a$

S	→	a B C	✗
S	→	a S B C	✗
B Y	→	B C	
C B	→	X B	
X B	→	X Y	
X Y	→	B Y	
a B	→	a b	✗
b B	→	b b	✗
b C	→	b c	✗
c C	→	c c	✗

NO está en FNK porque el lado derecho tiene un terminal seguido de dos no terminales (FNK permite A→BC, pero no A→**a**BC).

NO está en FNK (lado derecho tiene 4 símbolos y empieza con terminal).

Si en una gramática hay una regla que no está en FNK, ya se puede indicar que la gramática no está en FNK.
Podrían preguntarnos cuantas reglas incumplen la FNK ó motivos por los que incumple cada regla

No cumple FNK porque el lado izquierdo empieza con terminal (FNK no permite terminales en el lado izquierdo).

Reglas tomadas del apunte:
UTN, FRRO – Apunte de Cátedra – Capítulo 2: Gramáticas
https://www.frro.utn.edu.ar/repositorio/catedras/sistemas/2_anio/sintaxis/SySL_cap5_2015_Gram%C3%A1ticas.pdf

Conclusión

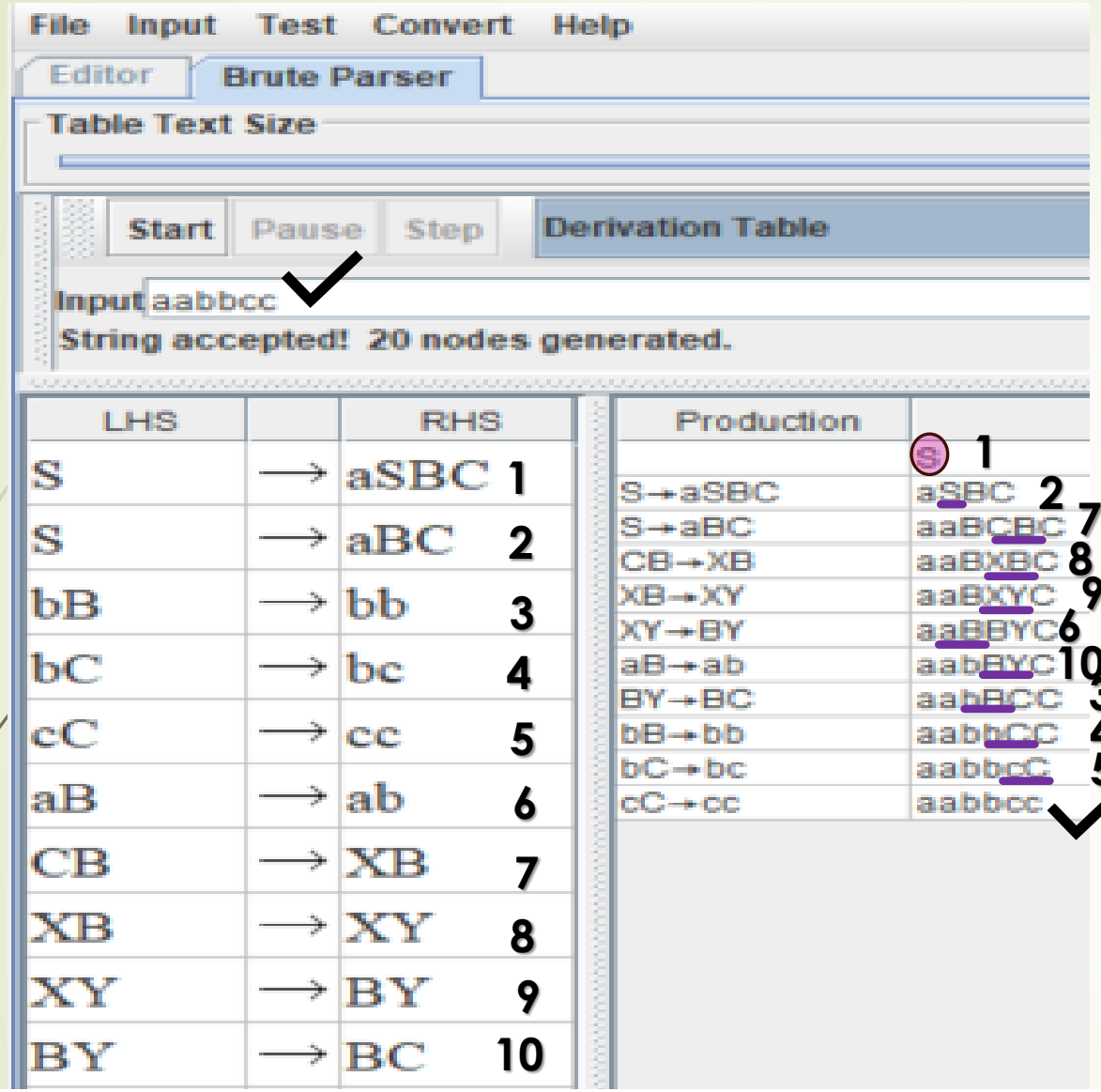
Si en una gramática hay una regla que no está en FNK, ya se puede indicar que la gramática no está en FNK.

Podrían preguntarnos cuantas reglas incumplen la FNK ó motivos por los que incumple cada regla

Ejemplo– Gramática: Lenguaje Tipo 1

JFLAP (v7) -> Imput -> Brute Force Parser

Para derivar la cadena: **aabbcc**



Reglas aplicadas:

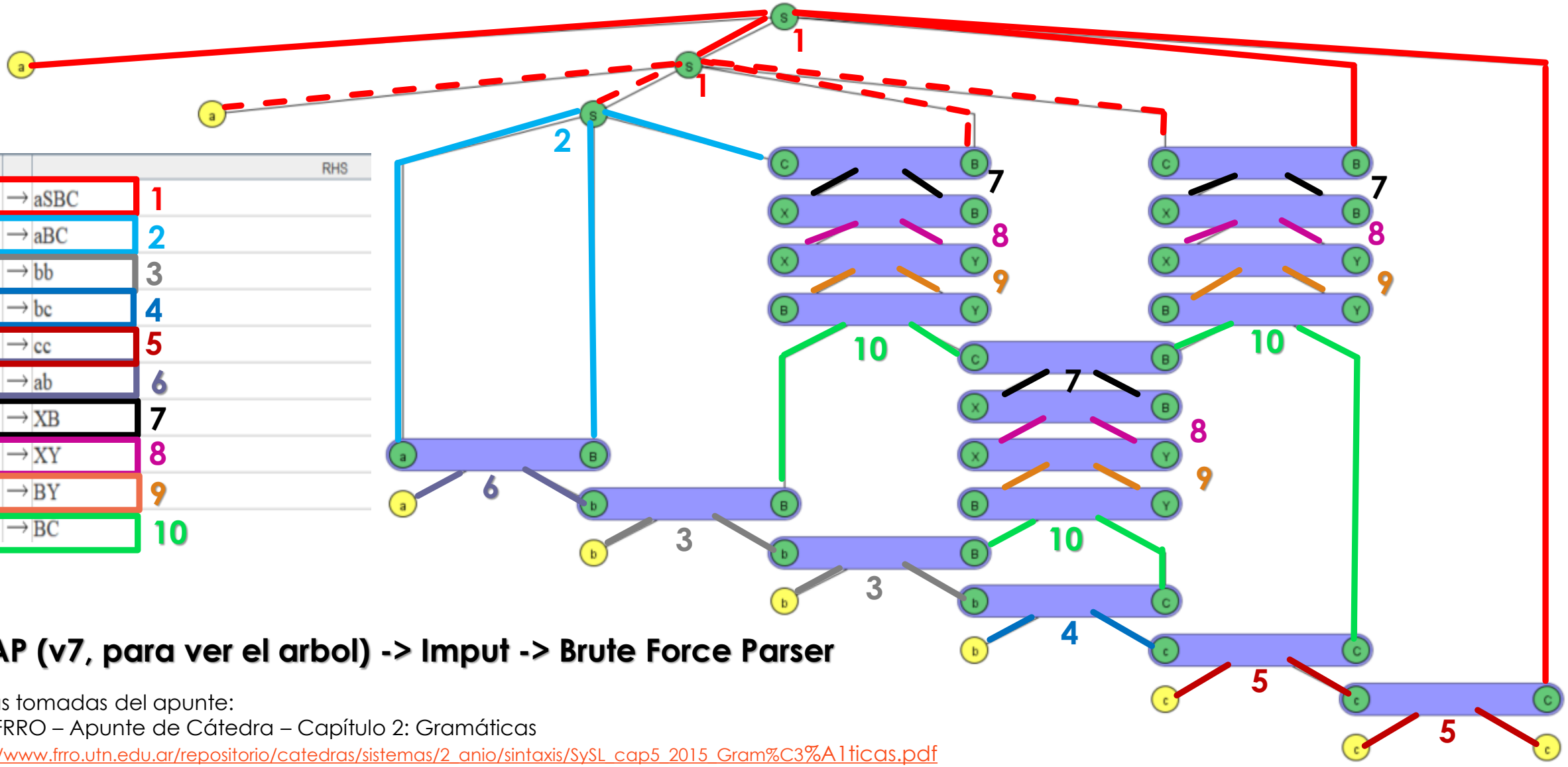
1 → 2 → 7 → 8 → 9 → 6 → 10 → 3 → 4 → 5

¿Que cambio del nivel 10 al 9?, pareciera que una C-> c Sin embargo esa regla no está!!! Que se hizo?

Reglas tomadas del apunte:
UTN, FRRO – Apunte de Cátedra – Capítulo 2: Gramáticas
https://www.frro.utn.edu.ar/repositorio/catedras/sistemas/2_anio/sintaxis/SySL_cap5_2015_Gram%C3%A1ticas.pdf

Ejemplo – Gramática: Lenguaje Tipo 1

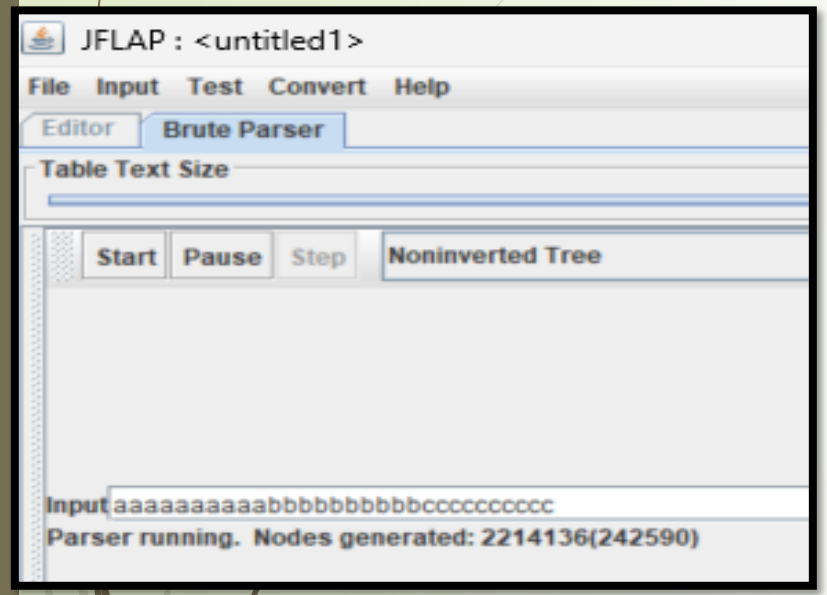
LHS		RHS	
S	→	aSBC	1
S	→	aBC	2
bB	→	bb	3
bC	→	bc	4
cC	→	cc	5
aB	→	ab	6
CB	→	XB	7
XB	→	XY	8
XY	→	BY	9
BY	→	BC	10

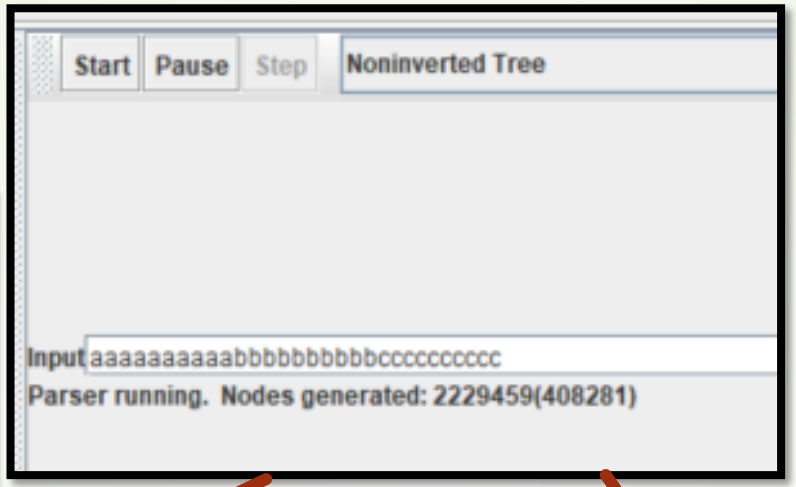


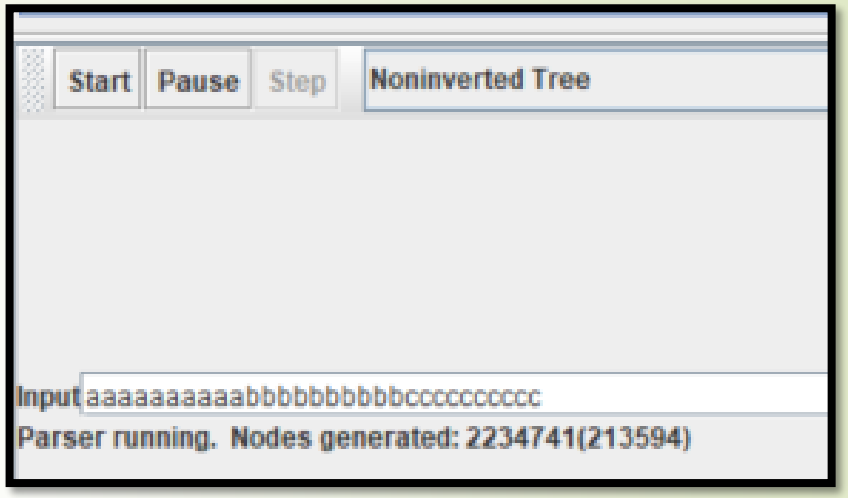
JFLAP (v7, para ver el arbol) -> Imput -> Brute Force Parser

Reglas tomadas del apunte:
UTN, FRRO – Apunte de Cátedra – Capítulo 2: Gramáticas
https://www.frro.utn.edu.ar/repositorio/catedras/sistemas/2_anio/sintaxis/SySL_cap5_2015_Gram%C3%A1ticas.pdf

Caso de Estudio - Complejidad en el análisis de gramáticas Tipo 1







2.229.459
total de nodos generados por el algoritmo de análisis
(todas las configuraciones exploradas).

(408.281): nodos actualmente en la lista activa (pendientes de expansión).



¿Por qué crece tanto?

- Cada símbolo puede interactuar con reglas sensibles al contexto que dependen de vecinos.
- **No hay poda ni optimización: el parser intenta todas las secuencias posibles.**
- El árbol de derivación se ramifica enormemente.

Conclusiones – Complejidad Gramáticas Tipo 1

Para una gramática Tipo 1, el reconocimiento es **PSPACE-completo** (requiere espacio polinómico en el tamaño de la entrada, pero tiempo puede ser exponencial).

El algoritmo de fuerza bruta en JFLAP es **mucho peor**: explora todas las derivaciones posibles, lo que implica: **Tiempo exponencial** (o incluso factorial) en la longitud de la cadena.

Explosión combinatoria: cada paso puede generar múltiples nuevas configuraciones. Por eso, para una cadena de longitud 10–12, se generan **millones de nodos**.

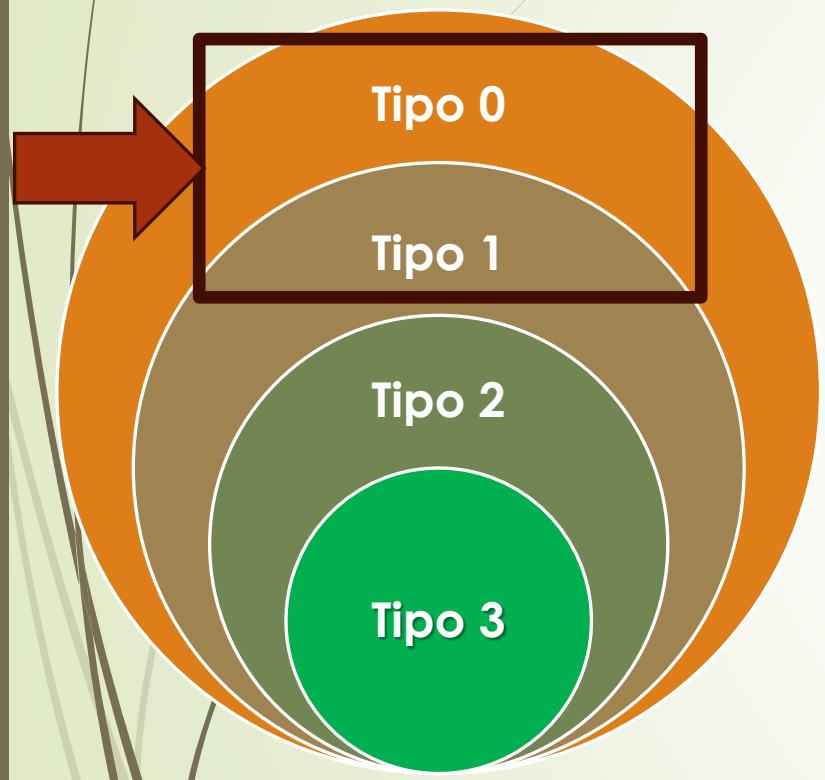
- Se usan simuladores y estrategias más eficientes (p. ej., autómatas linealmente acotados) para evitar esta explosión.
- Un compilador genera una tabla de símbolos, para tomar en cuenta su contexto (si un identificador corresponde a un entero).

Jerarquía de Chomsky (1956) – Lenguaje Tipo 0

Un lenguaje de Tipo 0 en la jerarquía de Chomsky también se conoce como:

- **Lenguaje sin restricciones** (porque no impone limitaciones en la forma de las reglas de producción).
- **Recursivamente Enumerable**, ya que corresponde al conjunto de lenguajes que pueden ser reconocidos por una Máquina de Turing (aunque no necesariamente decidibles).
- El término recursivamente numerable se refiere a un conjunto de cadenas (lenguaje) para el cual existe un algoritmo (en este caso, una Máquina de Turing que puede enumerar todas las cadenas que pertenecen al lenguaje). Esto implica:
  - Si una cadena pertenece al lenguaje, la máquina la aceptará (se detendrá y dirá "sí").
  - Si una cadena no pertenece, la máquina puede no detenerse nunca (es decir, no garantiza respuesta negativa en tiempo finito).
  - Por eso se dice que son reconocibles pero no necesariamente decidibles:
    - **Reconocible:** la máquina puede confirmar pertenencia.
    - **No Decidible:** no siempre puede confirmar no pertenencia.
- En otras palabras, son los lenguajes que pueden ser listados por un procedimiento computable, aunque no siempre podamos decidir la pertenencia de manera total.

Jerarquía de Chomsky



Lenguaje		Gramática	Máquina
Tipo 3	Lenguajes Regulares	GR	Autómata Finito (AF)
Tipo 2	Lenguajes Independientes de Contexto	GIC	Autómata de Pila
Tipo 1	Lenguajes Dependiente de Contexto	GDC	**Máquina de Turing (MT)** Autómatas Acotados
Tipo 0	Lenguajes sin Restricciones	Gramáticas Generales	**Máquina de Turing (MT)**

Máquina de Turing

- La diferencia entre los lenguajes sensibles al contexto y los recursivamente enumerables; está dada por el modelo de máquina de Turing: si la cinta está acotada por la longitud de la entrada (Tipo 1); si la cinta es ilimitada, obtenemos una MT general (Tipo 0).
  - En teoría, **sí se usan MT con cinta acotada** para caracterizar Tipo 1.
  - En la práctica, cuando trabajamos con ejemplos o construcciones, solemos usar directamente **MT sin cinta acotada**, porque es el modelo estándar y más general.
  - Por eso, en las cátedras y en la investigación, frecuentemente se habla de "máquina de Turing" sin aclarar la restricción, salvo cuando se quiere marcar la diferencia entre Tipo 1 y Tipo 0.

Inglés, 23 de junio de 1912 -1954
Lic. en Matemáticas
Dr. en Matemáticas

Campos de trabajo:

- Matemáticas y lógica matemática
- Criptografía y criptoanálisis
- Máquina de Turing
- Inteligencia artificial (IA)
- Biología matemática y morfogénesis

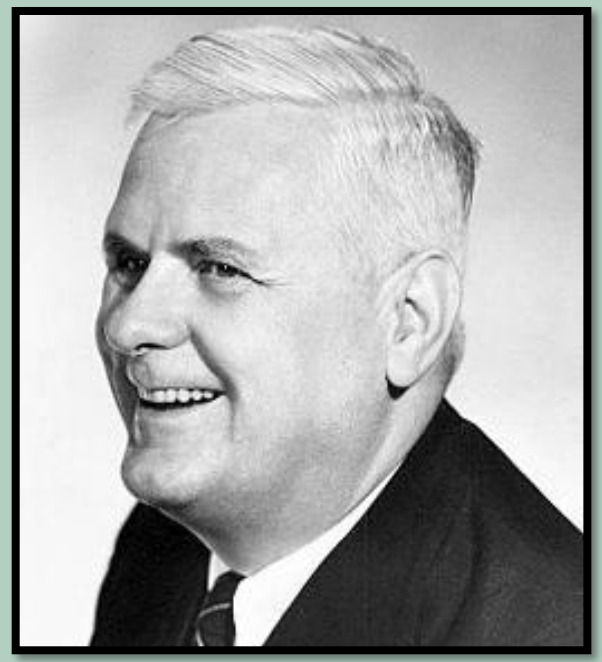


Alan Turing

Tesis de Church

- **Tesis de Church**: establece que todo problema que puede resolverse mediante un procedimiento efectivo (algoritmo) puede ser calculado por una Máquina de Turing.

La Jerarquía de Chomsky clasifica lenguajes según su complejidad y las máquinas que los reconocen. La Tesis de Church nos dice que esas máquinas representan todo lo que es computable en principio.



Alonzo Church

Nació en Estados Unidos
14 de junio de 1903 – 1995
Lic. en Matemáticas
Dr. en Matemáticas

Aportes destacados:

- Formulación del λ-cálculo
- Base teórica para la computación funcional.
- Tesis de Church (junto con Turing), que define el concepto de computabilidad efectiva.

Otros modelos matemáticos equivalentes a la Máquina de Turing

- Lambda Cálculo (Alonso Church)
- Funciones Recursivas (Kurt Gödel, Stephen Kleene)

Diferentes formalismos, misma potencia computacional.



Kurt Gödel

Nació en Austria
28 de abril de 1906 -1978
Lic. en Matemáticas
Dr. en Matemáticas

Aportes:

- Teoremas de Incompletitud: demostraron que en cualquier sistema formal suficientemente potente existen proposiciones verdaderas que no pueden ser demostradas dentro del sistema.
- Influencia en la noción de problemas indecidibles y límites de la computación.
- Base conceptual para la teoría de la computabilidad y la lógica moderna.

Máquina de Turing (MT)

Introducción. Antecedentes históricos

- "En la década de los 30, el inglés Alan Turing diseñó el modelo matemático de una máquina teórica con un gran poder computacional, llamada Máquina de Turing (M.T.)".
- "La gran ventaja de la M.T. es que, a pesar de su simplicidad, tiene un gran poder computacional y no se ve limitada por las características tecnológicas de una computadora como la velocidad de procesamiento o la capacidad de la memoria. Por esta razón, se la considera como un símbolo invariante de la informática".

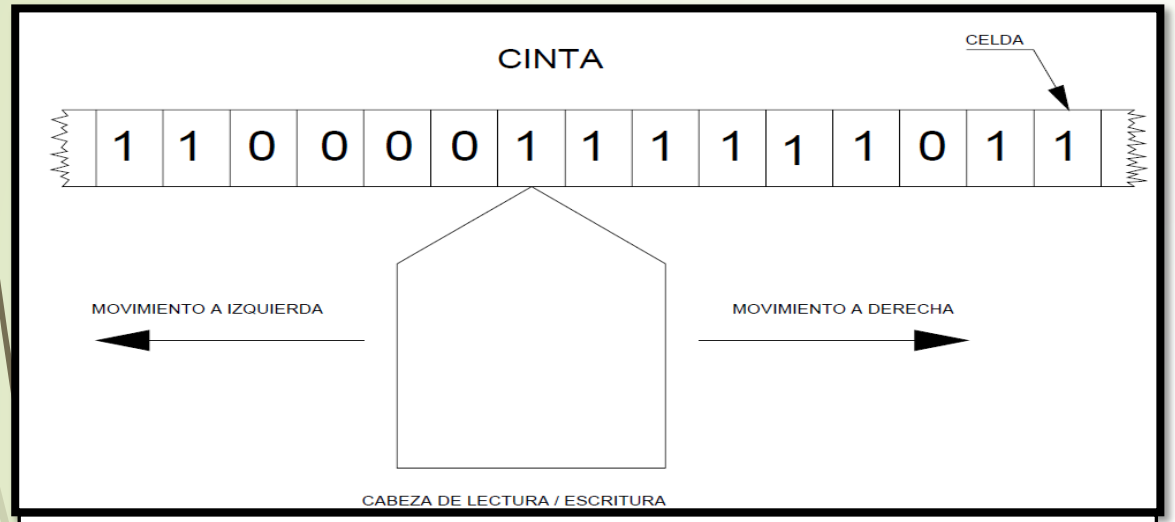


"La M.T. es un modelo matemático para representar a una máquina teórica. A pesar de su simplicidad la M.T. tiene el mismo poder computacional que una computadora de propósito general. La M.T. es interesante, sobre todo, por el conjunto de lenguajes que permite reconocer … (lenguajes recursivamente enumerables) y por el conjunto de funciones que puede computar (funciones calculables)".

Jurado Málaga, J. Teoría de autómatas y lenguajes formales. Universidad de Extremadura, Servicio de Publicaciones, 2008. ISBN 978-84-691-6345-0

https://dehesa.unex.es/entities/publication/51da7af5-d5fa-4472-b018-e93e4c375d54

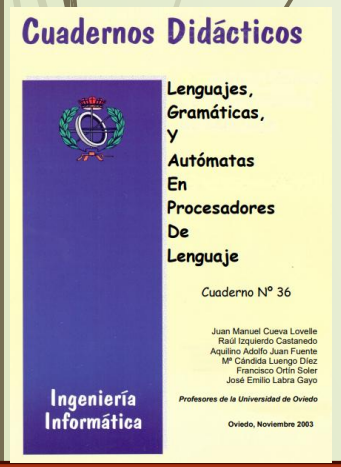
MT – Modelo General
Cinta Infinita-Lenguaje Tipo 0



En la figura, se representa el caso particular de un conjunto de símbolos en la cinta formados por 0 y 1.

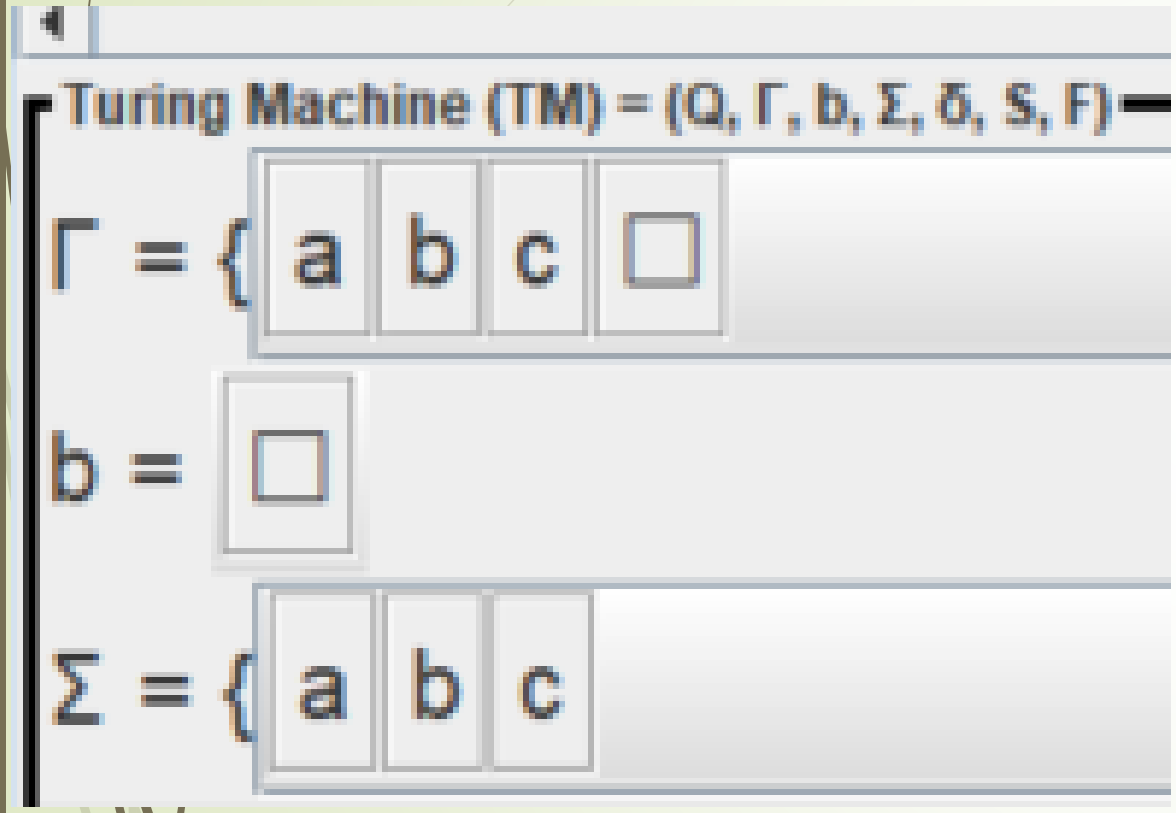
"Una máquina de Turing o autómata de tipo 0 es una construcción lógica, que se puede representar intuitivamente como un dispositivo mecánico, formado por una cinta infinita, dividida en celdas, y un cabezal de lectura/escritura que se mueve sobre dicha cinta, avanzando de a una celda por vez".

"Un movimiento de la máquina de Turing, depende del símbolo explorado con el cabezal y del estado actual en que se encuentra la máquina, el resultado puede ser: a) Cambio de estado; b) Escribe un símbolo en la cinta reemplazándolo por el símbolo leído; c) Se mueve la cabeza de la cinta a la izquierda, a la derecha o se para. Pueden darse los tres fenómenos anteriores, juntos o separados".



Lenguajes, gramáticas y Autómatas. Segunda Edición. Juan Manuel Cueva Lovelle, Raúl Izquierdo Castanedo, Aquilino Adolfo Juan Fuente, Cándida Luengo Díez, Francisco Ortín Soler, José Emilio Labra Gayo (Universidad de Oviedo, España, 2003)

MT – Cinta Infinita - Vista en JFLAP



Ejemplo – Vista en JFLAP

Q = {conjunto de estados}
Γ = {conjunto de símbolos permitidos en la cinta}
b = es el símbolo blanco (representado por un cuadrado vacío)
Σ = conjunto símbolos de entrada (no incluye el blanco)
δ = donde es la función del siguiente movimiento (L: Izquierda; R: Derecha; S: No se mueve)
S=estado inicial
F=conjunto de estados Finales o de aceptación.

MT – Autómata Linealmente Acotado Lenguaje Tipo 1

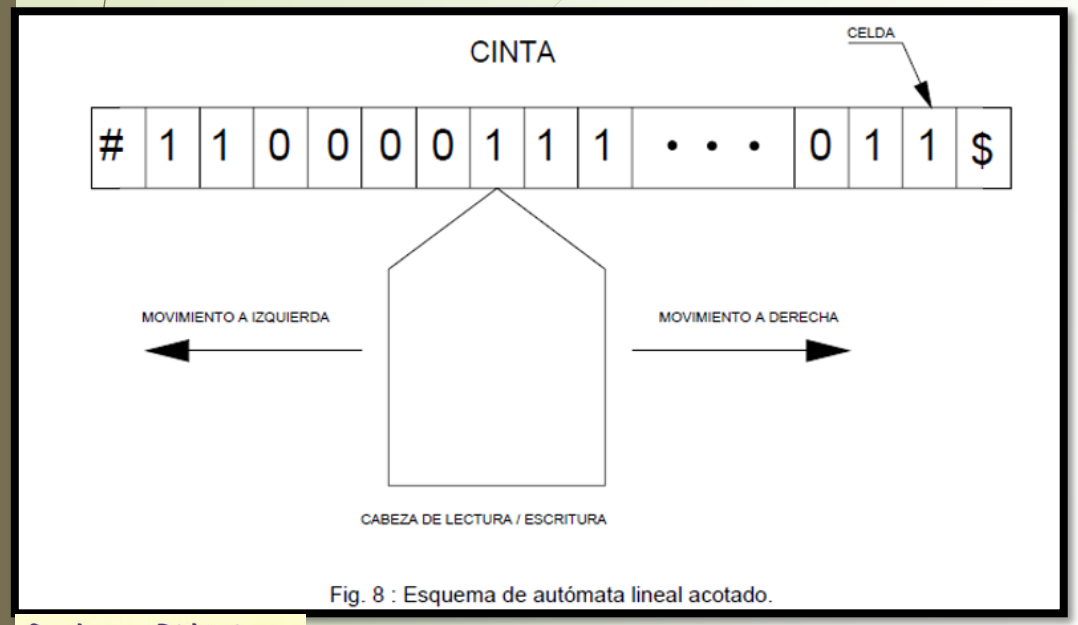
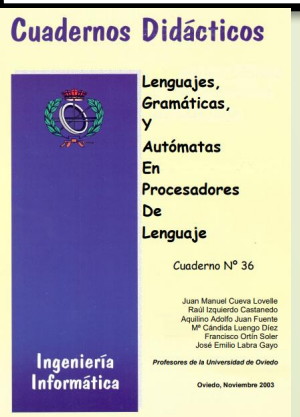


Fig. 8 : Esquema de autómata lineal acotado.

Los autómatas de tipo 1 son los autómatas limitados linealmente o autómatas lineales acotados (en inglés linear bounded automaton). Un autómata lineal acotado es una máquina de Turing que satisface las siguientes condiciones:

1. El alfabeto de entrada incluye dos símbolos especiales # y $, que son las marcas fin de cinta, izquierda y derecha respectivamente.
2. La cabeza del autómata no puede desplazarse fuera de los limites izquierdo y derecho de la cinta, y no puede imprimir otro símbolo sobre # y $.



Para toda gramática sensible al contexto G1 existe un autómata reconocedor lineal acotado RALA, tal que el lenguaje generado por la gramática L(G1), es reconocido por el autómata RALA.

Lenguajes, gramáticas y Autómatas. Segunda Edición. Juan Manuel Cueva Lovelle, Raúl Izquierdo Castanedo, Aquilino Adolfo Juan Fuente, Cándida Luengo Díez, Francisco Ortín Soler, José Emilio Labra Gayo (Universidad de Oviedo, España, 2003)

Esquemas: Una sóla cinta o Multicinta

- En el modelo clásico de Máquina de Turing de una sola cinta, esa cinta cumple varias funciones:
  - Contiene la entrada inicialmente escrita en la parte izquierda de la cinta.
  - Sirve como memoria de trabajo: la máquina puede leer, escribir y mover el cabezal sobre esa misma cinta.
  - Puede contener la salida: al finalizar, la cinta queda con el resultado (por ejemplo, aceptado, rechazado, o una cadena transformada).

Entonces, no es solo "cinta de entrada", sino cinta universal: entrada, procesamiento y salida ocurren en el mismo espacio.

En cambio, en una Máquina de Turing multicinta:

- Una cinta suele contener la entrada original (solo lectura).
- Las otras cintas se usan como espacio de trabajo adicional, lo que facilita ciertas operaciones (por ejemplo, copiar símbolos, comparar extremos, etc.).

Aunque esto mejora la eficiencia, no aumenta la potencia computacional: todo lo que hace una multicinta puede simularse en una sola cinta, aunque más lento.

Ejercicios Lenguajes

a) $L_1 = \{ a^n b^n c^n \quad n \geq 0 \}$

Ejercicios b y c de Tarea. Multicinta

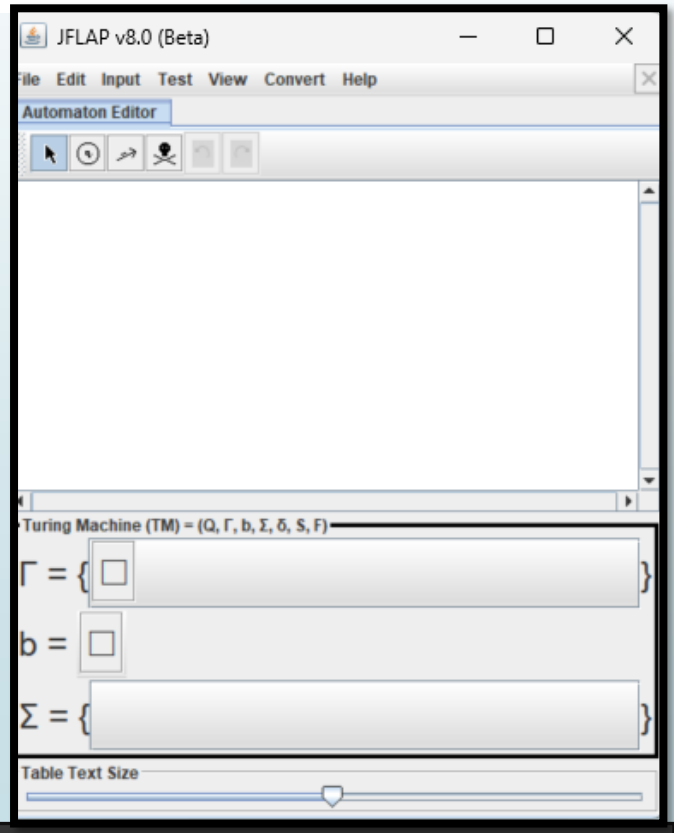
b) $L_2 = \{a^n b^m c^n d^m \quad n, m \geq 0\}$

c) $L_3 = \{a^n b^m c^p \quad n=m \text{ o } m=p;\ n, m \geq 0\}$ (dependencias condicionales complejas)

Salvo que se nos diga lo contrario los podemos resolver con multicinta (no más de 3 cintas en total)

Pasos a seguir:

1) Pensar la estrategia (puede haber más de una estrategia que resuelva el mismo problema)
2) Armar el autómata en FJLAP e ir testeando su funcionamiento



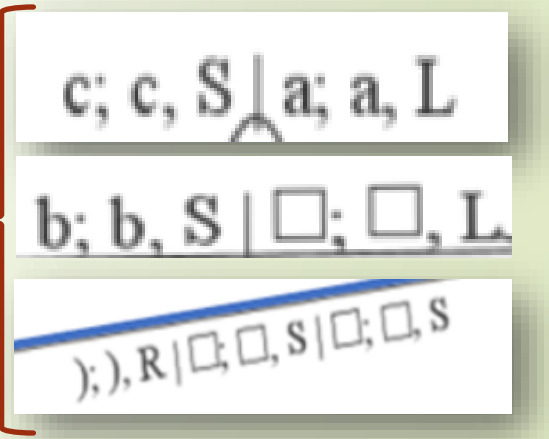
Notación JFLAP

Carácter que leo; Carácter que escribo, Movimiento

- Si leo y escribo lo mismo, se interpreta como que dejé lo que estaba en la cinta
- Si el carácter que leo ó escribo es blanco se utiliza un cuadrado 

- Movimientos posibles:
  - S: No se mueve
  - R: Se mueve a la derecha
  - L Se mueve a la izquierda

Ejemplos multicinta



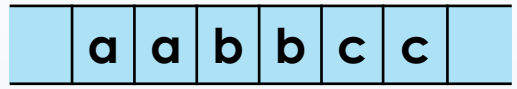
EJERCICIOA_R1.jflap

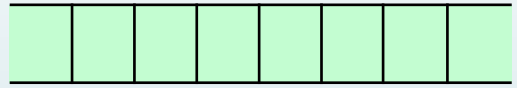
Resolución 1 - Ejercicio a

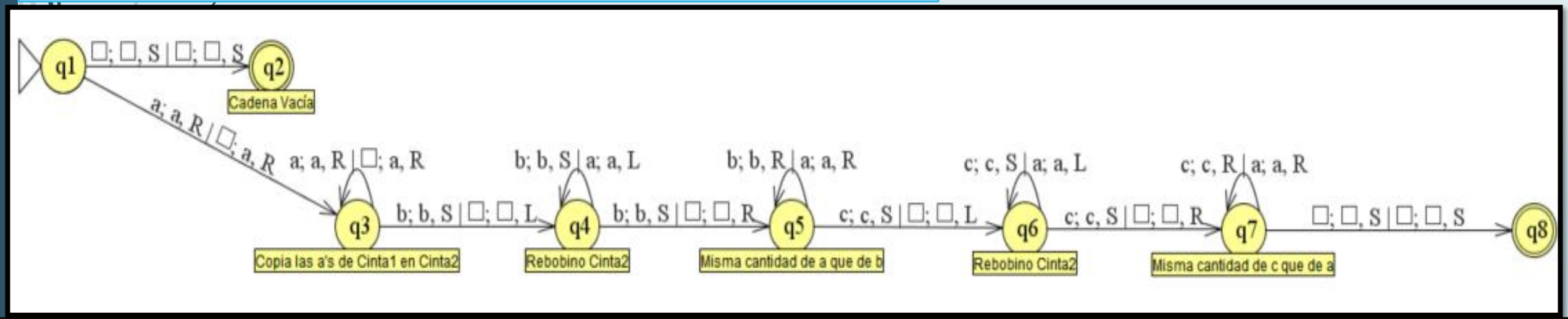
a) $L_1 = \{ a^n b^n c^n \ n \geq 0 \}$

Estrategia 1:
Cinta 1: Cadena Original
Cinta 2: Cinta auxiliar de trabajo

Copiar las a's de la cinta 1 a la cinta 2
"Rebobinar" la cinta 2, recorrer cada b de la cinta 1, desplazándome por cada una de ellas en la cinta 2, si al llegar a la última b me queda la última a en la cinta 2 (es porque la cantidad es igual de a que de b). Repito el mismo proceso para la c.







EJERCICIOA_R2m.jflap

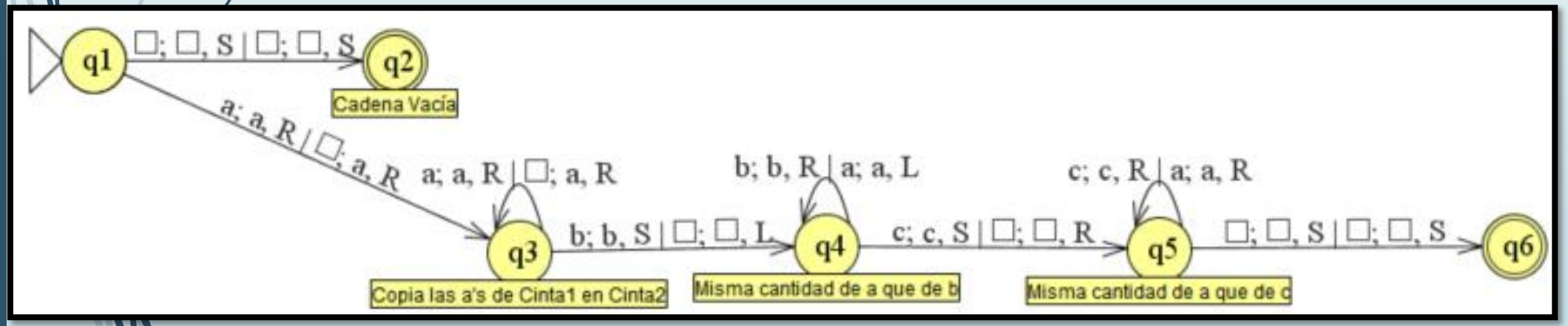
Resolución 2 - Ejercicio a

a) $L_1 = \{ a^n b^n c^n \quad n \geq 0 \}$

Dos estados menos!!

Estrategia 1:
Cinta 1: Cadena Original
Cinta 2: Cinta auxiliar de trabajo

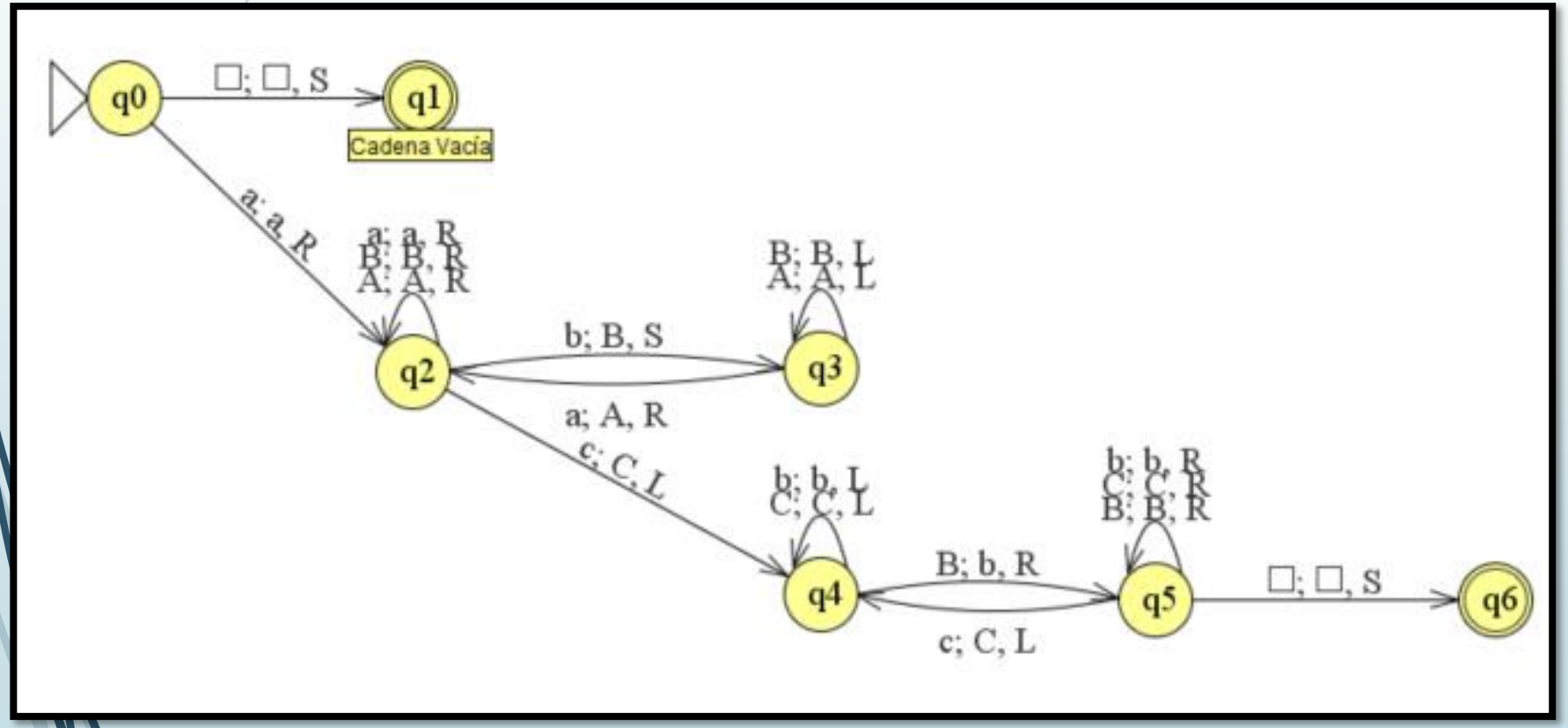
Copiar las a's de la cinta 1 a la cinta 2
Por cada b (cinta 1) retrocedo una a (cinta 2) (cuando se acaban las b tengo que tener blanco en cinta 2)
Por cada c (cinta 1) avanzo una a (cinta 2) (cuando se acaban las c tengo que tener blanco en cinta 2)



Lo cargamos en JFLAP

Resolución 3 – Ejercicio a - En una sola cinta

Recorrer la cinta original por cada b minúscula reemplazar por una B mayúscula y cambiar una a minúscula por una A mayúscula. Luego por cada c minúscula reemplazar por una C mayúscula y cambiar A una a minúscula.



¡Lo vemos funcionando!

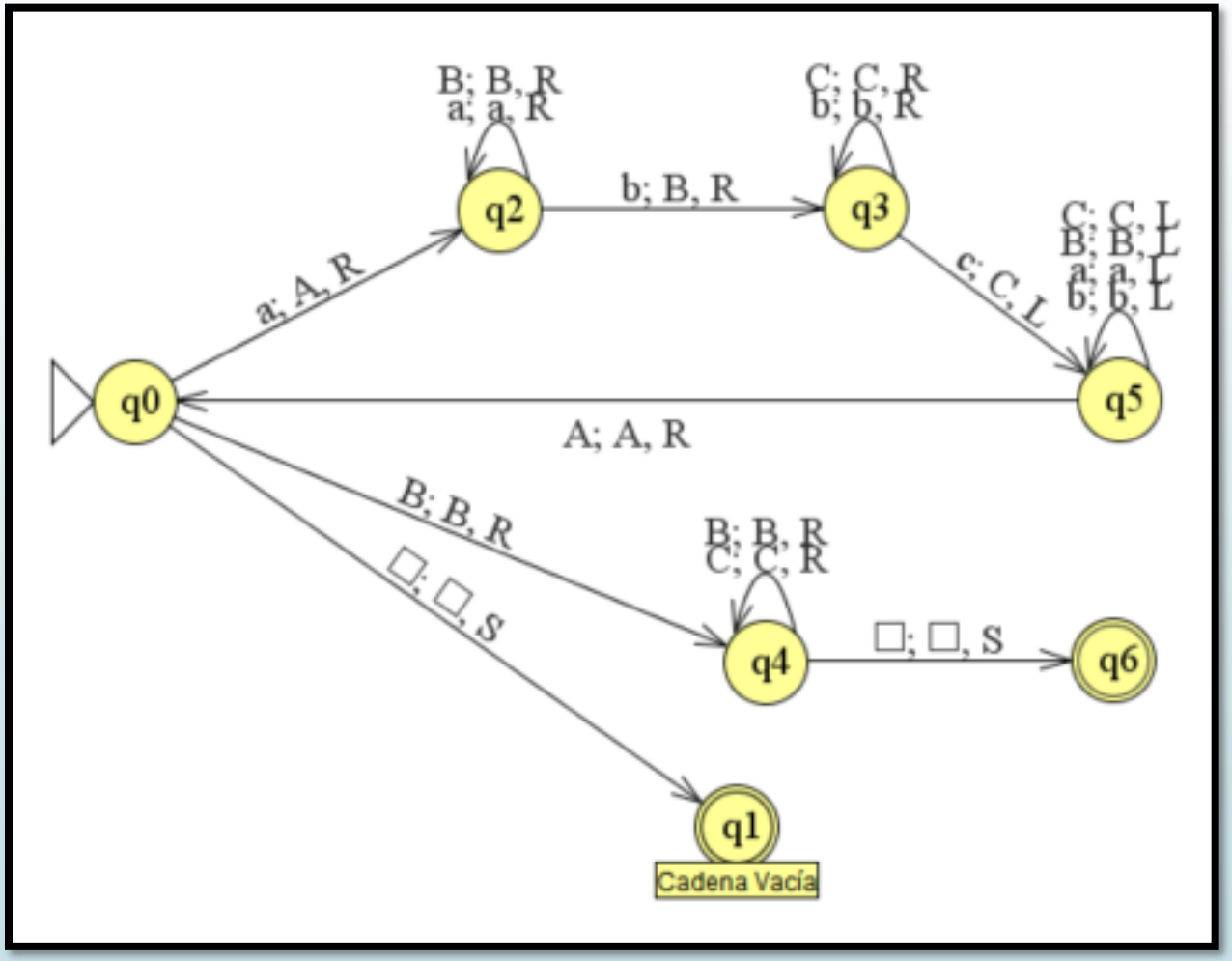
Resolución 4 – Ejercicio a - En una sola cinta

Paso 1: Se busca la primera a, se reemplaza por una A, luego se busca la primera b y se reemplaza por una B y luego se busca la primera c y se reemplaza por una C

Paso 2: se vuelve a la izquierda hasta la primera A y se desplaza a la derecha

Se repite el paso 1 hasta que no haya más a para reemplazar

¡Lo vemos funcionando!



Pasos… Importancia de una estrategia

Análisis

- **Leer, comprender e interpretar correctamente el enunciado**.
- En esta etapa se identifican los requerimientos del problema.
- ¿Qué recibe la máquina como entrada, qué debe producir o reconocer y cuáles son las condiciones que debe cumplir la solución?

Diseño (Estrategia)

- ¿Cuántas cintas se utilizarán?
- ¿Para qué se utilizará cada cinta?
- ¿Qué información será necesario conservar o marcar?
- ¿Cómo se organizará el proceso de resolución?
- ¿Qué pasos generales deberá seguir la máquina?
- En esta etapa todavía **no se construyen los estados ni las transiciones**. Define la idea de solución que posteriormente será traducida al autómata.

Desarrollo (Construcción)

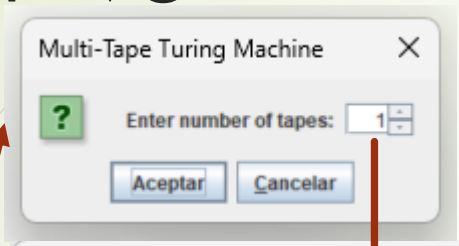
- Se lleva la estrategia diseñada a una Máquina de Turing concreta.
- Se definen **los estados, las transiciones, los símbolos, los movimientos de los cabezales**, etc.

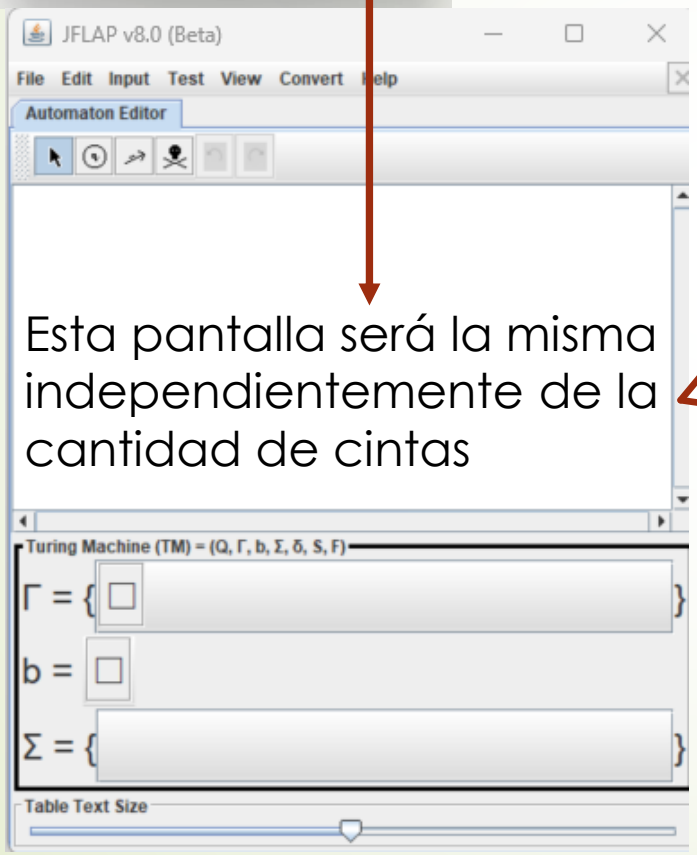
Pruebas y Validación

- **Lote de pruebas:** debe incluir los casos representativos de las distintas situaciones de interés planteadas por el enunciado, contemplando: casos normales, casos límite y situaciones que puedan presentar dificultades.
- **Comportamiento:** comprobar que la MT funcione correctamente y responda de manera esperada frente a los diferentes tipos de entradas.
- **Validación:** verificar que la MT cumpla con los requerimientos definidos durante el análisis y que la estrategia planteada se haya implementado correctamente.

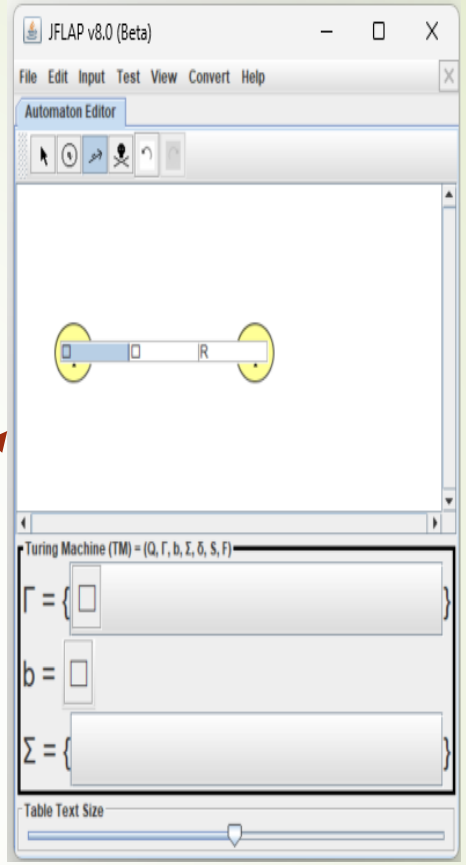
JFLAP v8



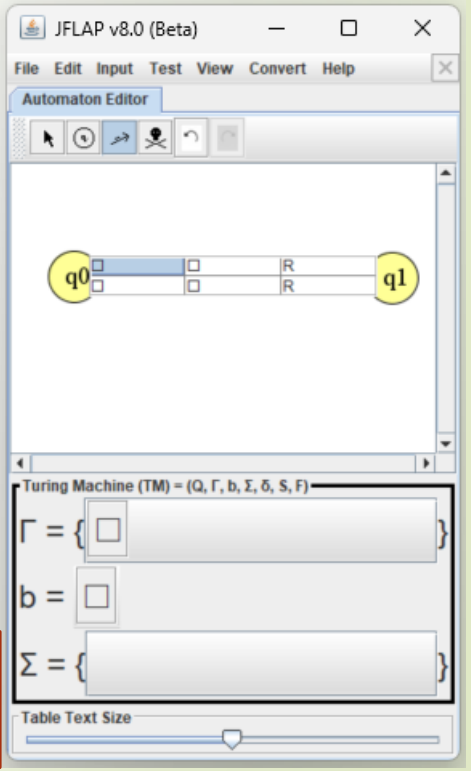




Esta pantalla será la misma independientemente de la cantidad de cintas



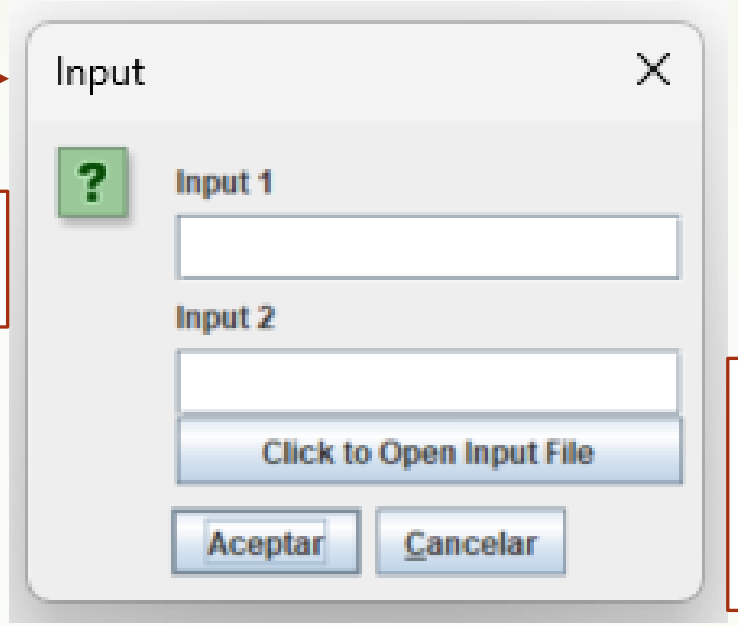
Vista si se configuró una cinta



Vista si se configuró dos cinta

JFLAP v8

Imput -> Step by Step



Input 1 contenido de la cinta 1 en la configuración inicial

Nosotros tomaremos las otras cintas como vacías inicialmente, entonces no llenamos nada en: Input 2, Input 3…

Configuración Inicial y Final de la Cinta



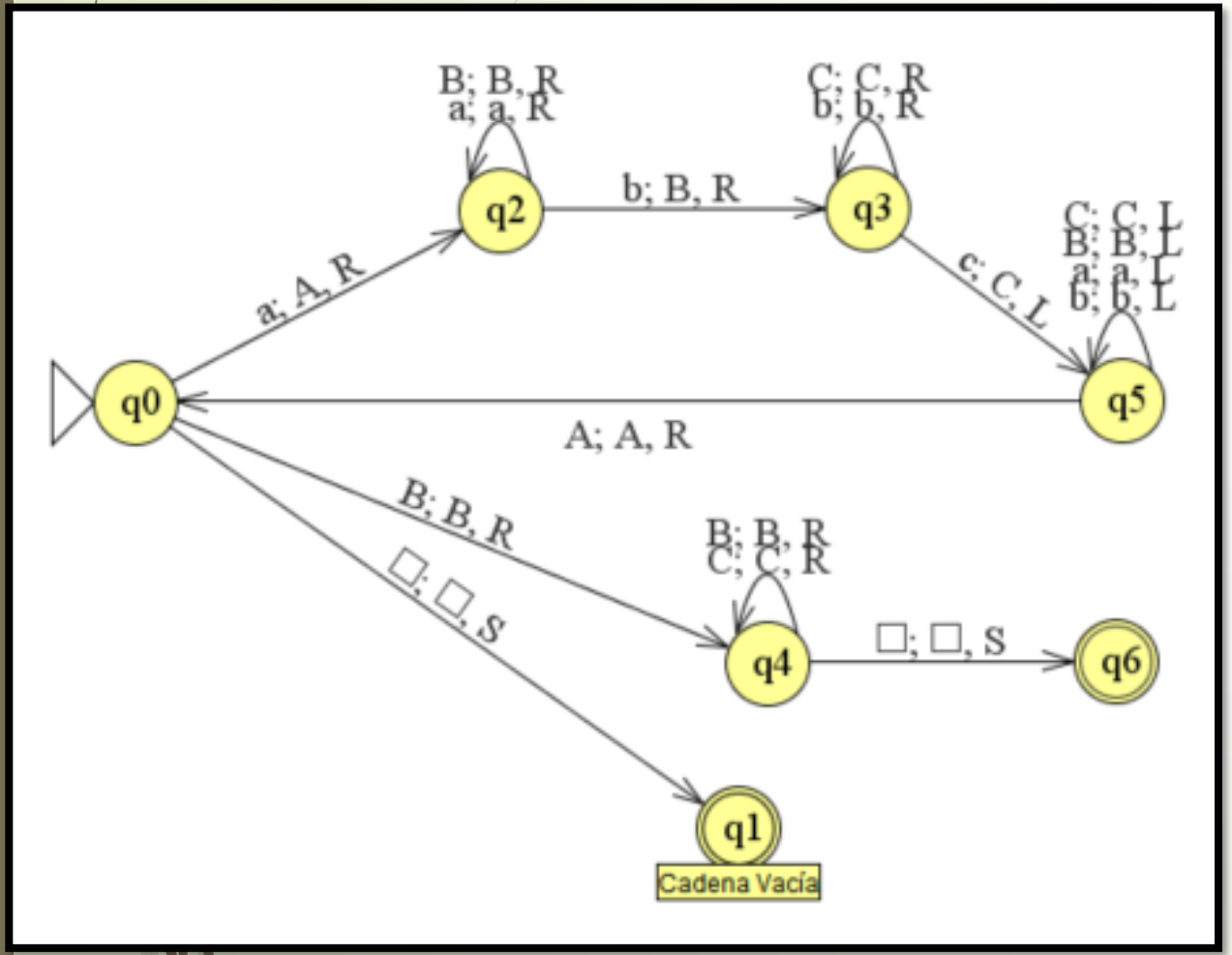
Configuración Inicial: Puede ser que el ejercicio me pida mantener intacta la configuración inicial de la cinta, esto implicaría por ejemplo "rebobinar" la cinta y reemplazar las mayúsculas puestas durante el proceso por minúsculas



Configuración Final: También más allá de llegar a un estado de aceptación o de rechazo al detectar una cadena, podrían pedirme dejar en la cinta un =0 cuando es rechazada ó un =1 cuando es aceptada la cadena.

MT - Tabla de Transiciones

δ		Cintas		Estado Destino
Estado Origen	C1	C1		
		Símbolo	Mov.	



δ: La función de transición toma en cuenta el estado actual (origen) y el contenido de la cinta.

Si es **MT multicinta** en la parte celeste (Cintas) deberían estar representadas cada una de las cintas C1, C2…

Estado Destino, al que se llega al aplicar esa transición

δ		Cintas		Estado Destino
Estado Origen	C1	C1		
		Símbolo	Mov.	
q0	a	A	R	q2
q0	B	B	R	q4
q0	□	□	S	q1
q1	-	-	-	-
q2	a	a	R	q2
q2	B	B	R	q2
q3	C	C	R	q3
q3	b	b	R	q3
q3	c	C	L	q5
q4	B	B	R	q4
q4	C	C	R	q4
q4	□	□	S	q6
q5	C	C	L	q5
q5	B	B	L	q5
q5	a	a	L	q5
q5	b	b	L	q5
q5	A	A	R	q0
q6	-	-	-	-

Determinismo en MT (MTD ó ~~MTND~~) - Unicinta

PARA UNA MT UNICINTA:

Si al leer un determinado carácter hay más de una posibilidad en cualquiera de estos 3 componentes:

Lo que se escribe

Por ejemplo, si se lee A y en la cinta se escribe X en una rama; Y en otra rama.

Hacia dónde se mueve el cabezal

Una transición podría moverlo a la izquierda (L) y otra a la derecha (R).

Próximo Estado

Al leerse un símbolo A se escribe una B en X en la cinta y se mueve a la derecha pero hay dos transiciones exactamente iguales que llevan a estado de destino distintos.

Conceptualmente, la máquina "explora" todas las opciones en paralelo.
Una cadena se acepta si al menos una de las ramas llega a un estado de aceptación.

Determinismo en MT (MTD ó ~~MTND~~) - Multicinta

PARA UNA MT MULTICINTA:

Para una misma configuración (estado actual y símbolos leídos en cada cinta), la máquina puede tener varias transiciones posibles.

- **Lo que se escribe en cada cinta**
- **Hacia dónde se mueve el cabezal en cada cinta**
- **Cuál es el Próximo Estado**

Igual que en la unicinta, una cadena se acepta si **al menos una rama** llega a un estado de aceptación.

38 Ejercicio Propuesto (enunciado Mara Capuya)

Se tiene una función **map** que toma una operación [+numero] y una lista de números y devuelve una lista entre corchetes con la operación +numero aplicada a cada uno de los números de la lista. Si la lista original está vacía, devuelve [].

El formato de la función es: **map** [**+numero; (Lista de números)**] donde *Lista de números* es una lista de números entre paréntesis separados por comas y el número que acompaña al operador +, es un número mayor o igual a cero.

La operación +numero y la lista están separados por un punto y coma.

Ejemplos: map [+2; (1,20,13)] devuelve [3,22,15]
map [+3; (5,2,3)] devuelve [8,5,6]
map [+1; ()] devuelve []

Todos los números se encuentran en notación unaria. Los corchetes, la palabra *map*, la coma, el signo + y el punto y coma son parte de la función.

Configuración inicial: map [+11 ; (1,11,1111)]

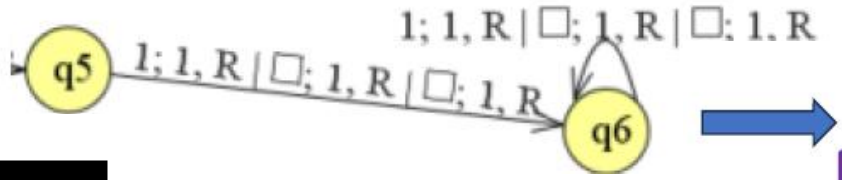
Configuración final: map [+11 ; (1,11,1111)] = [111,1111,111111]

Tareas:

- Escribir la estrategia para diseñar una MT para computar la función MAP.
- Desarrollar la MT.
- Probarlo en JFLAP (lote de pruebas).

Comienzo: Siempre que hay partes fijas que serán iguales en todos los casos, simplemente es pasar entre estados.





Estrategia

map [+11;(1,11,1111)]	
11	Resultado
11	Auxiliar

Estrategia 1:

1) Copiar el número en Cinta 2 y Cinta 3
2) El primero de la lista agregará sus 1 a la Cinta 2
3) Pongo la coma en Cinta 2
4) Copio lo de la Cinta 3 a la Cinta 2
5) El segundo de la lista agregará sus 1 a la Cinta 2
6) Pongo la coma en Cinta 2
7) Copio lo de la Cinta 3 a la Cinta 2
8) El tercero de la lista agregará sus 1 en la Cinta 2

¿Y si la lista está vacía?

La cinta de resultado ya quedó con contenido. No sirve esta estrategia!

Estrategia

map [+**11**;(1,11,1111)]

Resultado

11

Estrategia 2:

1) **Copiar el número en Cinta 3**
2) El valor de la lista agregará sus 1 a la Cinta 2
3) Concateno Cinta2 con el número de Cinta 3
4) Pongo la coma en Cinta 2

- El **)** marca el fin de la lista que puede estar vacía **()**
- Al llegar hasta **]** en Cinta 1:
  - ir al inicio de la Cinta 2
  - agregar en la Cinta 1 **=[**
  - copiar lo de la Cinta 2
  - y poner en la Cinta 1]



¿Y si la lista está vacía?

¿Si no viene nada en Numero a sumar?

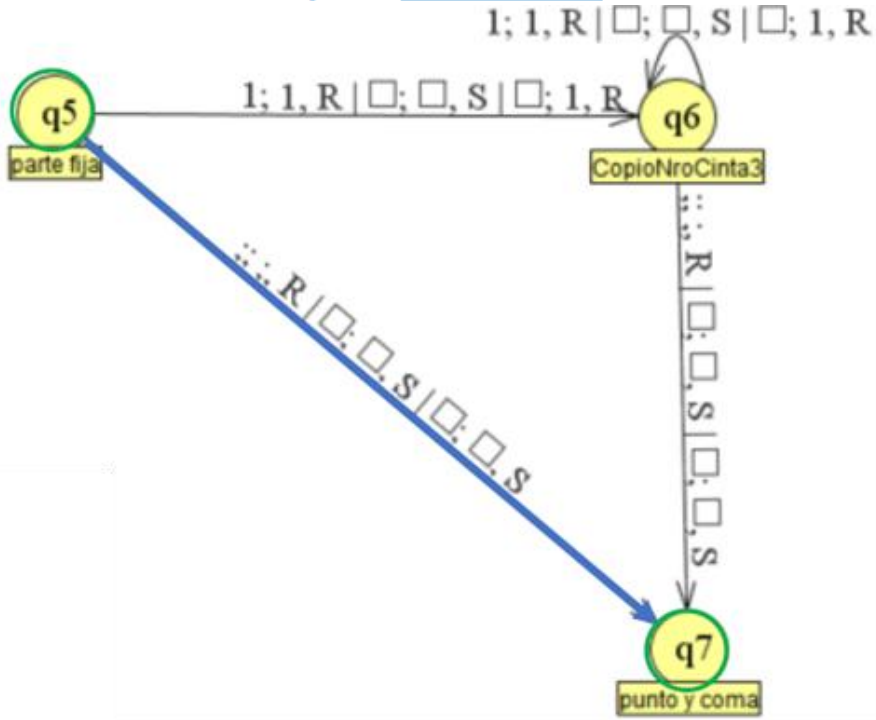
¿Si vienen más de tres valores o menos de tres?



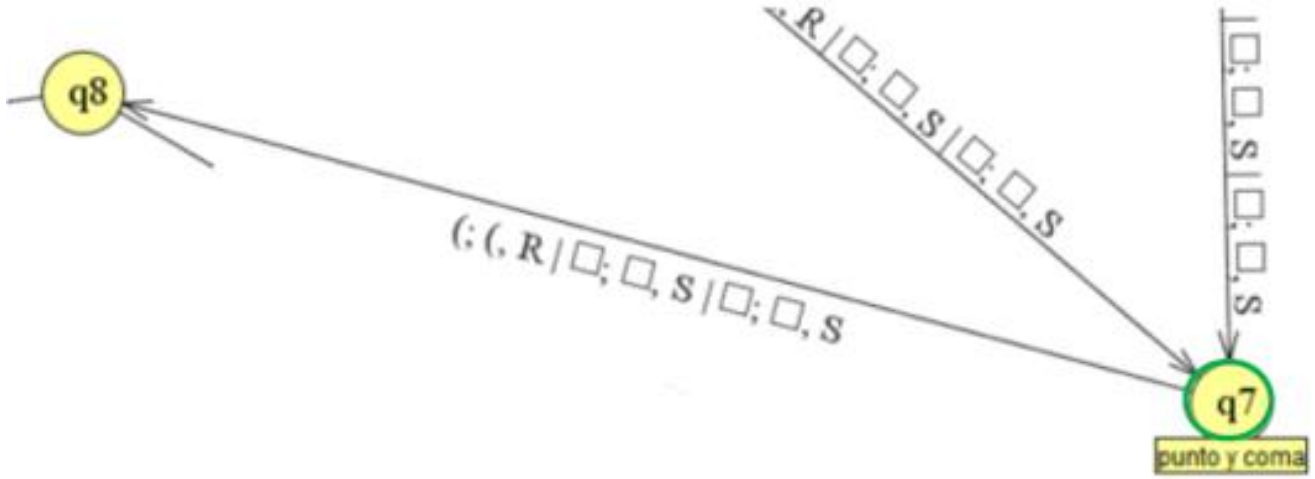

```
map [+11;(1,11,1111)]
```

Va a estar siempre, es parte del formato de la función

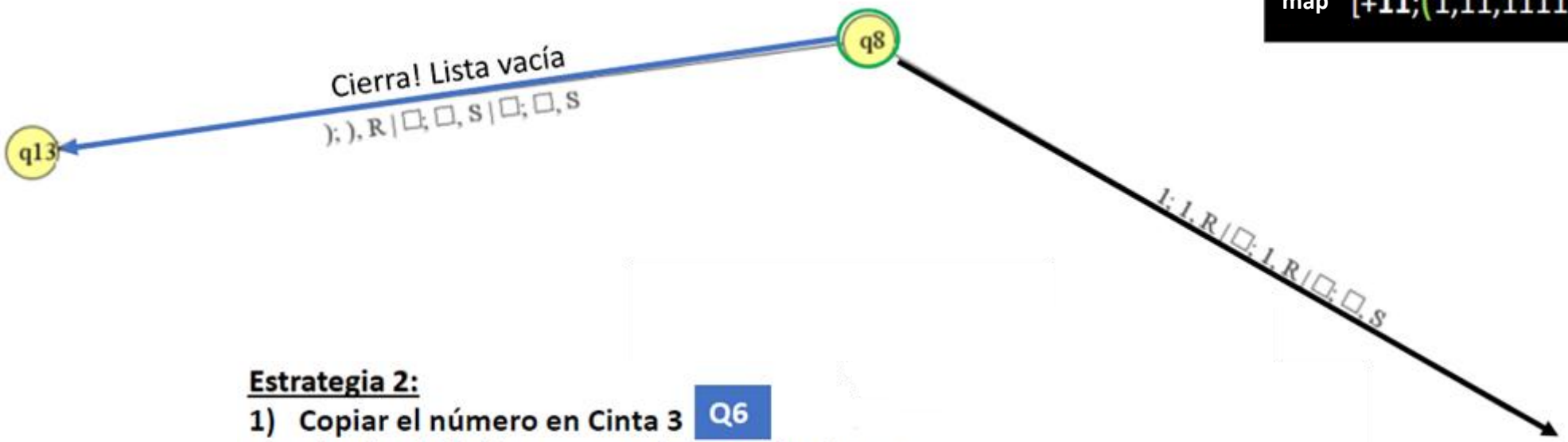
El enunciado de Mara nos dice que puede llegar un número unario mayor o igual a cero



Una vez que llegó el ; debe haber un paréntesis que abre



map [+11;(1,11,1111)]



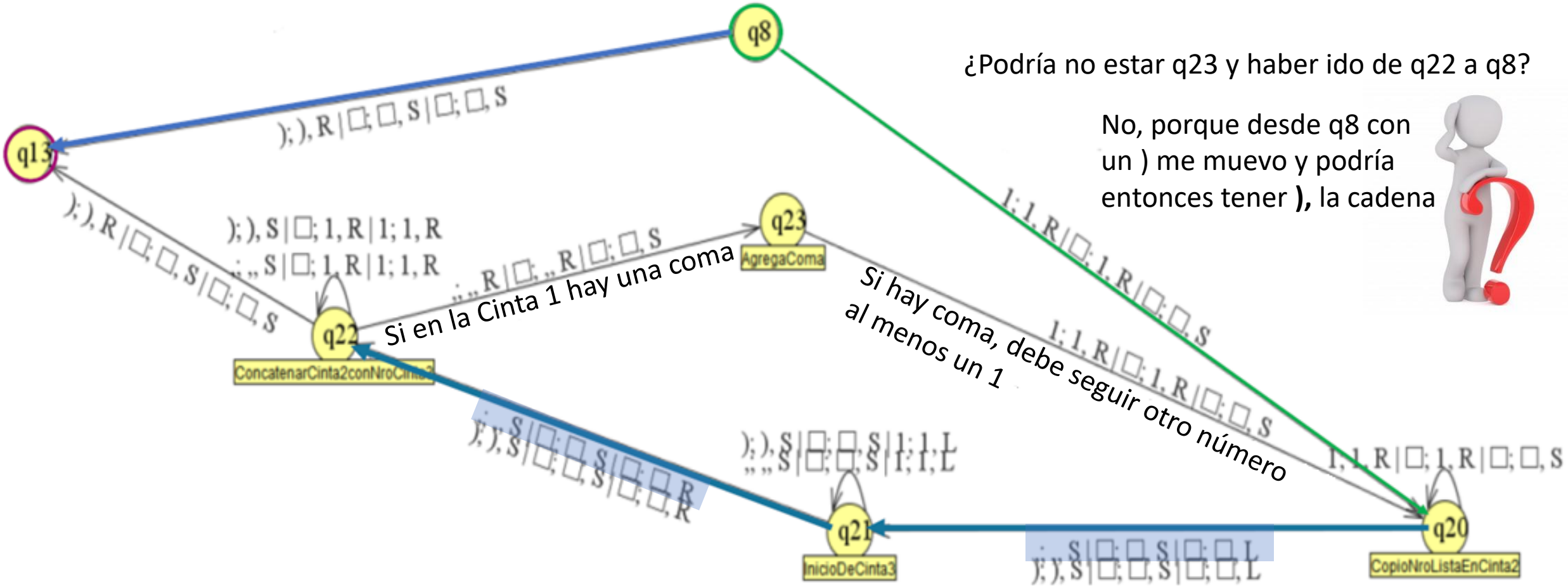
Estrategia 2:

1) **Copiar el número en Cinta 3** Q6
2) **El valor de la lista agregará sus 1 a la Cinta 2**
3) Concateno Cinta2 con el número de Cinta 3
4) Pongo la coma en Cinta 2

Vemos que hay un Ciclo o repetición

- El **)** marca el fin de la lista que puede estar vacía **()**
- Al llegar hasta **]** en Cinta 1:
  - ir al inicio de la Cinta 2
  - agregar en la Cinta 1 **=[**
  - copiar lo de la Cinta 2
  - y poner en la Cinta 1]

map [+11;(1,11,1111)]



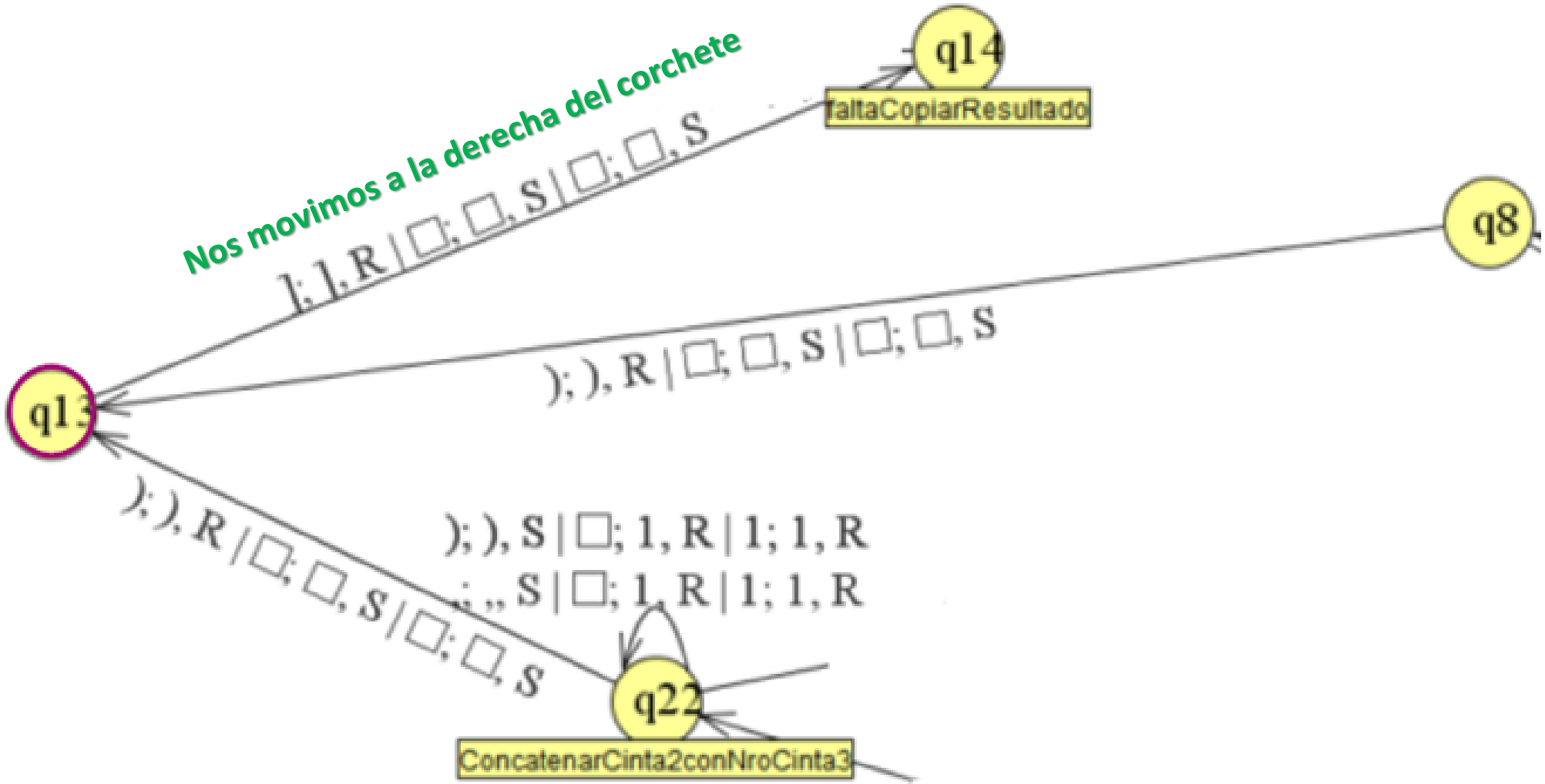
¿Podría no estar q23 y haber ido de q22 a q8?

No, porque desde q8 con un) me muevo y podría entonces tener **),** la cadena

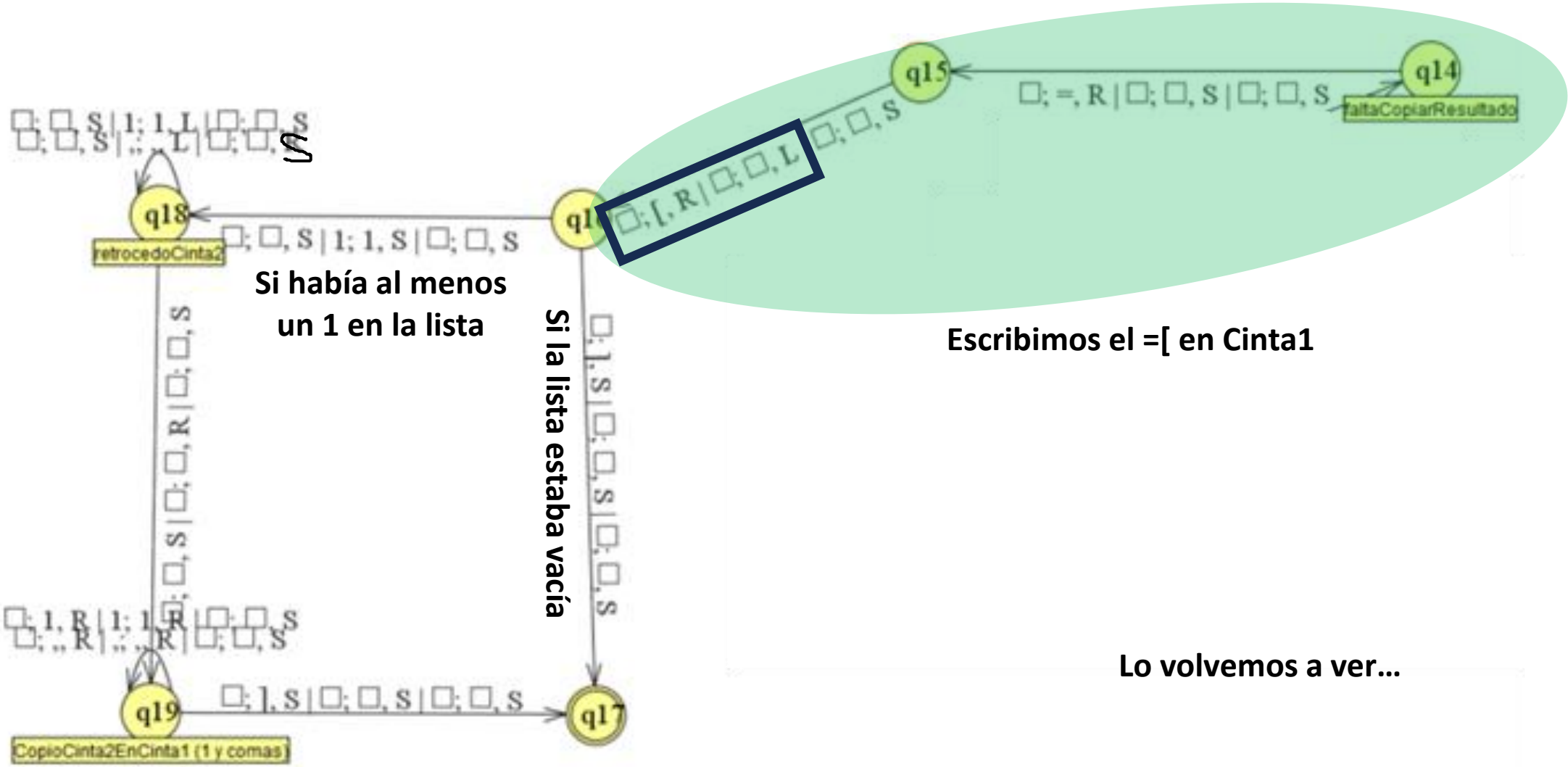
Coma si habrá más números en la lista
o paréntesis si es el último

map [+11;(1,11,1111)]

¿Qué hay en la cinta 1 en q14?

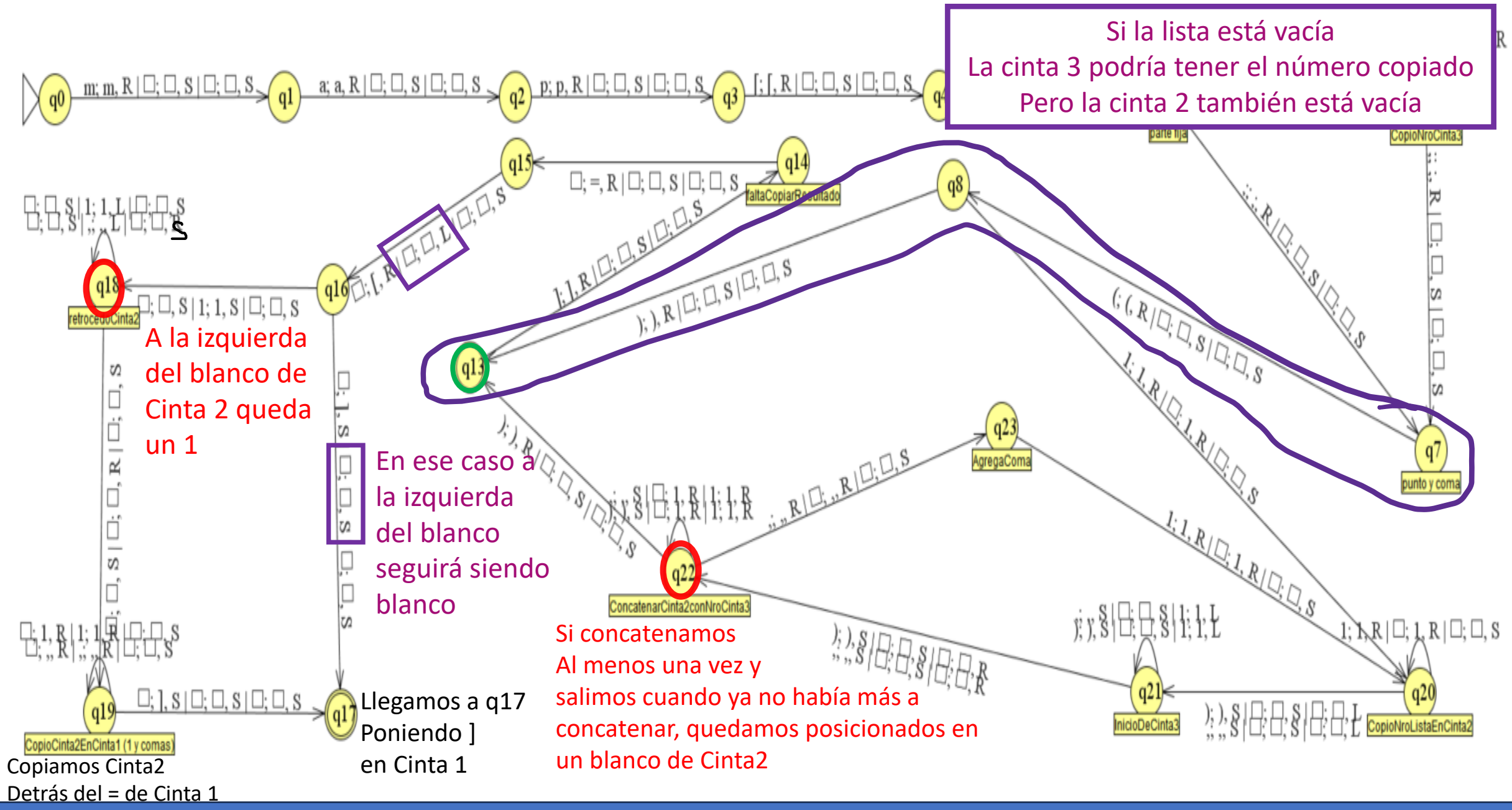


map [+11;(1,11,1111)]



Escribimos el =[en Cinta1

Lo volvemos a ver…


Si la lista está vacía
La cinta 3 podría tener el número copiado
Pero la cinta 2 también está vacía
A la izquierda del blanco de Cinta 2 queda un 1
En ese caso a la izquierda del blanco seguirá siendo blanco
Si concatenamos
Al menos una vez y
salimos cuando ya no había más a concatenar, quedamos posicionados en un blanco de Cinta2
Llegamos a q17
Poniendo]
en Cinta 1
Copiamos Cinta2
Detrás del = de Cinta 1
q0
q1
q2
q3
q7
punto y coma
q8
q13
q14
faltaCopiarResultado
q15
q16
q17
q18
retrocedoCinta2
q19
CopioCinta2EnCinta1 (1 y comas)
q20
CopioNroListaEnCinta2
q21
InicioDeCinta3
q22
ConcatenarCinta2conNroCinta3
q23
AgregaComa
CopioNroCinta3

Lo vemos funcionando

Lote de prueba:
map[+111; (1,11,111)]
map[+111; (1)]
map[+111; (1,11,11,111)]
map[+; (1,11,111)]
map[+111; ()]
map[+; ()]

MTPractica4.jflap

Computabilidad, Complejidad y Máquinas de Turing

Lenguajes Turing Completo

- Un sistema (lenguaje de programación, máquina, formalismo) es Turing completo si tiene la misma potencia computacional que una Máquina de Turing.
- Esto implica que puede simular cualquier algoritmo computable, siempre que disponga de recursos ilimitados (tiempo y memoria).

- Características Básicas:
  - Condicionales (decisiones: if/else).
  - Iteración o recursión (bucles o llamadas recursivas).
  - Memoria ilimitada (en teoría).
- Relación con la Jerarquía de Chomsky:
  - Los lenguajes Turing completos están en el nivel más general (Tipo 0), porque pueden reconocer cualquier lenguaje recursivamente enumerable.

Ejemplos - Lenguajes Turing Completo

- **SON TURING COMPLETO**
  - C
  - C++
  - JAVA
  - PHYTON



¿Assembler es Turing Completo?

NO SON TURING COMPLETO

- Lenguajes de consulta limitados
  - SQL básico (sin procedimientos almacenados ni ciclos): puede realizar consultas, pero no ejecutar lógica general.
- Lenguajes de etiquetado:
  - HTML, CSS: describen estructura y estilo, pero no tienen condicionales ni bucles.
- Lenguajes de configuración:
  - JSON, YAML: solo definen datos, sin lógica de control.
- Lenguajes de dominio específico:
  - Por ejemplo, lenguajes para hojas de cálculo simples (sin macros), que no permiten recursión ni estructuras de control completas.

Aplicaciones que toman como entrada una MT

Automata-lib (Python): Biblioteca para definir y ejecutar MT. https://pypi.org/project/automata-lib

UTME (Universal Turing Machine Emulator): Emulador en Python para MT. https://github.com/utme-project

TuringMachineSimulator.com: Simulador online con notación declarativa (lenguaje reducido propio). https://turingmachinesimulator.com

Proyectos Open Source: Ejemplo: *Turing-Machine-Compiler* (Python). https://github.com/topics/turing-machine

NO EVALUABLE

Lenguajes Computables

Computable: significa que existe un algoritmo (o una MT) que puede procesar el problema/lenguaje.

Tipo 1 →

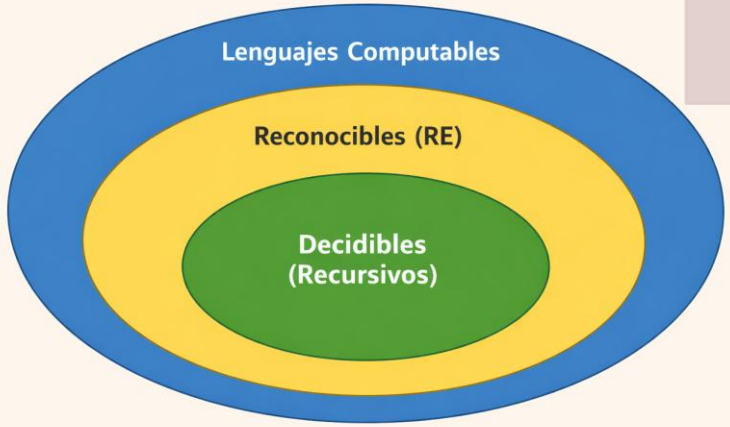
Decidibles (Lenguajes Recursivos)

- Una MT siempre se detiene y responde "sí" o "no" correctamente.

Tipo 0 ↓

Reconocibles (Lenguajes Recursivos Enumerables)

- Una Máquina de Turing acepta las cadenas correctas, pero si la cadena no pertenece, puede no detenerse nunca.
- Por eso, todos los decidibles son reconocibles, pero no todos los reconocibles son decidibles.



Decidible: Tengo un algoritmo que siempre me dice "sí" o "no".
Reconocible: Tengo un algoritmo que me dice "sí" si pertenece, pero puede quedarse en loop (ciclo infinito) si no pertenece.

Decidibles (Recursivos) ⊂ Reconocibles (RE) ⊂ Lenguajes computables

Complejidad MT

Una vez que sabemos qué el problema es computable, surge la pregunta: ¿cuánto cuesta resolverlos? Aquí entra la complejidad.

Problema Decidible

- Existe una MT que siempre para.

Problema Indecidible

- No existe un algoritmo, puede existir un proceso (MT puede no parar).

1. Concepto General

- La complejidad de una Máquina de Turing se refiere a los recursos necesarios para resolver un problema:
  - Tiempo: número de pasos que realiza la máquina.
  - Espacio: cantidad de celdas de cinta utilizadas.

2. Clases de Complejidad

Clase		¿Qué implica pertenecer a la clase?	Recurso que limita	Idea clave
P	Polynomial Time	Existe una **MT determinista** que resuelve el problema en **tiempo polinomial**	Tiempo	Problemas resolubles eficientemente
NP	Nondeterministic Polynomial Time	Existe una **MT no determinista** que lo resuelve en tiempo polinomial **o** una MT determinista que **verifica** una solución en tiempo polinomial	Tiempo (verificación)	Resolver puede ser difícil, verificar es fácil
PSPACE	Polynomial Space	Existe una MT (determinista o no) que lo resuelve usando **espacio polinomial**, sin límite sobre el tiempo	Espacio	Importa la memoria, no el tiempo
EXPTIME	Exponential Time	Todo algoritmo determinista requiere **tiempo exponencial** en el peor caso	Tiempo	Computable pero intratable en la práctica

P ⊆ NP ⊆ PSPACE ⊆ EXPTIME

3. Relación con la Máquina de Turing

- Una Máquina de Turing sirve como modelo para definir estas clases.
- **Tiempo y espacio** se miden en función del tamaño de la entrada.
- Ejemplo: Algoritmos que requieren $O(n^2)$pasos vs. $O(2^n)$pasos.

¡Recordamos!
Unidad 2 – Parte B

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(n^k) < O(2^n) < O(m^n) < O(n!)$$

Computabilidad

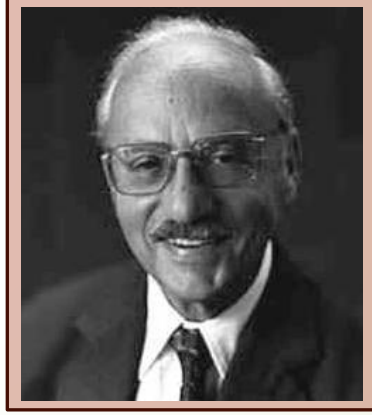
Demostración

Problema	Descripción	Autor / Año
Problema de la parada (Halting Problem)	Dado un programa y una entrada, decidir si el programa se detendrá o quedará en bucle infinito.	Alan Turing, 1936
Teorema de Rice	Determinar si un programa tiene una propiedad no trivial (ej. "calcula una función total", "siempre devuelve un número primo").	Henry Gordon Rice, 1953
Problema de la mortalidad de matrices	Dado un conjunto de matrices enteras, decidir si existe una secuencia de multiplicaciones que produzca la matriz cero.	Michael Paterson, 1970 (y refinamientos posteriores)
Hilbert's Entscheidungsproblem	Decidir si una fórmula de lógica de primer orden es universalmente válida.	Alonzo Church, 1936; Alan Turing, 1936
Problema de las ecuaciones diofánticas generales (Hilbert X)	Decidir si una ecuación polinómica con coeficientes enteros tiene soluciones enteras.	Yuri Matiyasevich, 1970 (sobre la base de trabajos de Davis, Putnam y Robinson)

EJEMPLOS

- **Por dentro de la jerarquía de Chomsky:** Todo lo que es computable
- Los problemas irresolubles
  - en el momento que puedan resolverse se pueden ubicar dentro de la jerarquía de Chomsky.

George Bernard Dantzig

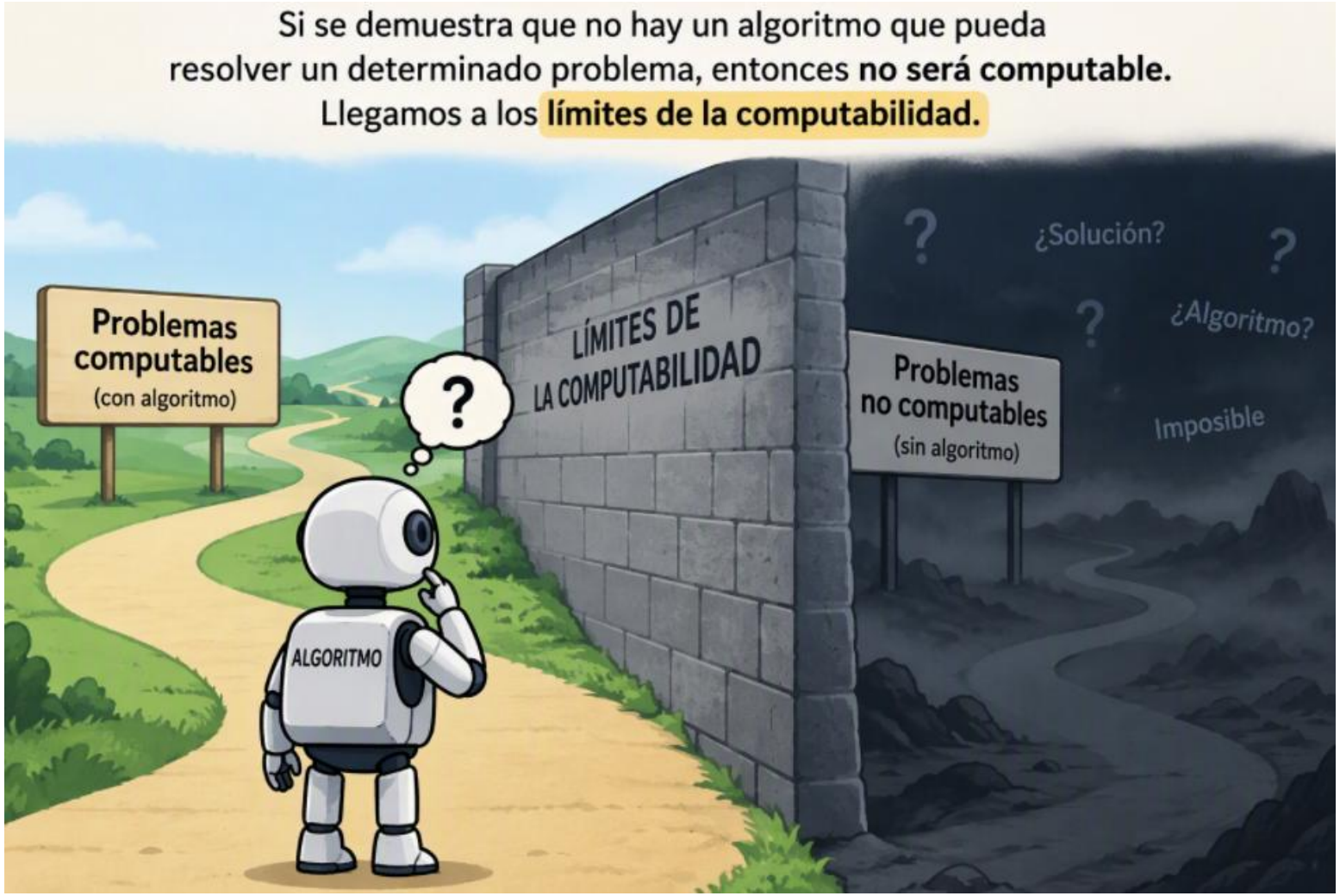


Nació en Estados Unidos
8 de noviembre de 1914 - 2005

Licenciado en Matemática y Física
Maestría en Matemática
Doctorado en Estadísticas

1939 – Estudiante de Doctorado

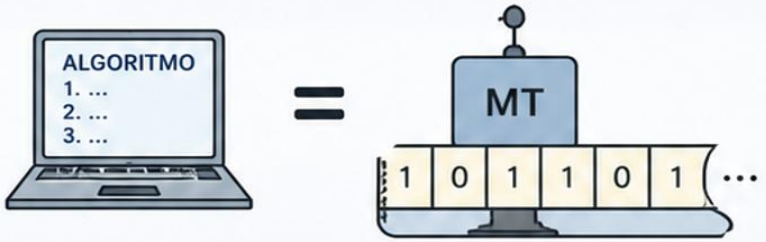
- **Por fuera corresponde a lo no computable:** problemas para los cuales no existe algoritmo que resuelta completamente el problema. Y ha sido probado matemáticamente.
  - No tiene que ver con la potencia de la máquina
  - No tiene que ver con el tiempo de resolución


Si se demuestra que no hay un algoritmo que pueda
resolver un determinado problema, entonces no será computable.
Llegamos a los límites de la computabilidad.
Problemas computables
(con algoritmo)
?
LÍMITES DE LA COMPUTABILIDAD
ALGORITMO
Problemas no computables
(sin algoritmo)
?
¿Solución?
?
?
¿Algoritmo?
Imposible

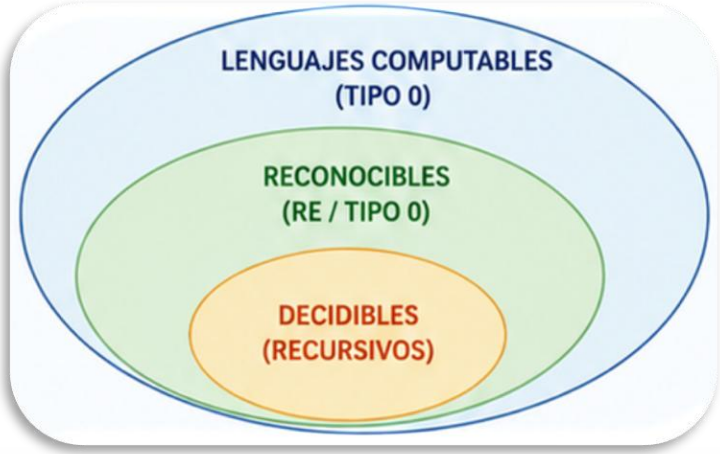
Resumen de Conceptos

COMPUTABLE

Un problema o lenguaje se considera computable si existe un procedimiento efectivo (algoritmo) o una Máquina de Turing (MT) capaz de procesarlo.



Según la **Tesis de Church-Turing**, las MT representan todo lo que es computable en principio.



RECONOCIBLE

(LENGUAJES RECURSIVAMENTE ENUMERABLES / TIPO 0)

Son lenguajes sin restricciones en sus reglas de producción.

Si la cadena pertenece	Si la cadena no pertenece
La MT se detiene y acepta.	La MT puede no detenerse nunca, entrando en un bucle infinito.
ACCEPT ("sí")	(no se detiene)

Por ello, se dice que son **reconocibles pero no necesariamente decidibles.**

DECIDIBLE

(LENGUAJES RECURSIVOS)

Es un subconjunto de los lenguajes reconocibles.

Existe un algoritmo que SIEMPRE se detiene en tiempo finito, proporcionando una respuesta correcta ("sí" o "no") tanto si la cadena pertenece como si no.

Si la cadena pertenece	Si la cadena no pertenece
ACCEPT ("sí")	REJECT ("no")

Decidibles ⊂ Reconocibles ⊂ Lenguajes Computables

FUERA DE LA JERARQUÍA (INCALCULABLES)

Lo que queda fuera son los problemas incalculables, para los cuales se ha probado matemáticamente que no existe un algoritmo general.

ACCEPT: la MT acepta y se detiene — REJECT: la MT rechaza y se detiene — Bucle infinito: la MT no se detiene

Relación entre la Jerarquía de Chomsky y la computabilidad



La Jerarquía de Chomsky, no sólo clasifica lenguajes por complejidad, sino que también refleja los límites de lo que es computable.



Todo lo que es computable, puede resolverse con un proceso ó algoritmo está por dentro de la Jerarquía de Chomsky y puede (a lo sumo como un lenguaje de Tipo 0), resolverse con una Máquina de Turing.



Lo computable entonces no depende de la tecnología, sino de que exista un proceso ó un algoritmo que pueda resolverlo y si existe entonces habrá una MT equivalente que puede resolverlo.

Fuerza bruta... un ejemplo...

Para una contraseña de 16 caracteres de longitud, con **74 caracteres posibles** (26 mayúsculas + 26 minúsculas + 10 números + 12 símbolos especiales)

Es Decidible

Espacio total de búsqueda – Por fuerza bruta:

- **Alfabeto:** 74 caracteres
- **Longitud:** 16 posiciones
- **Combinaciones totales:** 74^{16} = 808.551.180.810.136.214.718.004.658.176

Intratable - Impracticable

Cálculos de NotebookLM

Entorno de Cómputo	Capacidad de Procesamiento	Tiempo Estimado
CPU Estándar	100 millones de intentos/seg	256 billones de años(2,56 x 10^{14} años)
Clúster de GPUs de Alta Gama	10.000 millones de intentos/seg	2,56 billones de años (2,56 x 10^{12} años)
Supercomputadora Dedicada	1 billón (10^{12}) de intentos/seg	25.600 millones de años

https://www.ibm.com/mx-es/think/topics/quantum-cryptography

La MT Cuántica y los inalterables límites de la computabilidad

1981 –La idea ¿Porqué necesitamos computación cuántica?



Richard Feynman
11/05/1918-1988
Estados Unidos
Lic. en Física
(MIT, 1939)
Dr. Física
(Princeton, 1942)

1985 – El modelo – MT Cuántica



David Deutsch
08/12/1953
Reino Unido
Lic. en Cs. de la Computación
(Oxford/Cambridge)
Dr. Física Teórica
(Oxford, 1978)

1994 – El potencial – Algoritmo Short, que cambia la criptografía



Peter Shor
14/08/1959
Estados Unidos
Lic. en Matemáticas
(Caltech, 1981)
Dr. Matemáticas Aplicadas (MIT, 1985)

2000 – El límite – La computación cuántica no cambia la indecibilidad de los problemas



Scott Aaronson
21/05/1981
Estados Unidos
Lic. en Ciencias de la Computación
Cornell University, 2000)
Dr. en Ciencias de la Computación
(UC Berkeley, 2004)

(demostró que las computadoras cuánticas no resuelven el Halting Problem).

Ejemplos “cotidianos”

- **DECIDIBLE**
  - Adivinar por fuerza bruta la contraseña de 16 caracteres (letras, símbolos, números). **Impracticable/Intratable**
  - El sistema de pago que debe indicar si alcanza el dinero en la cuenta bancaria para poder hacer una compra.
  - Saber si la probabilidad de un choque en una fecha y horario exacto es mayor al 10%
- **RECONOCIBLE**
  - ¿Queda algún ejemplar de pájaro carpintero? Alguien va caminando con un celular ve uno y saca una foto, entonces el sistema puede comparar la foto y decir que sí hay. Pero porque alguien no haya subido la foto no implica que sea un animal en extinción.
- **NO COMPUTABLE**
  - Se quiere saber si en una fecha y horario exacto habrá un choque en la puerta de la universidad

Vamos a la presentación de Federico y después regresamos…

Máquinas y sus Autores
MT como punto de Partida

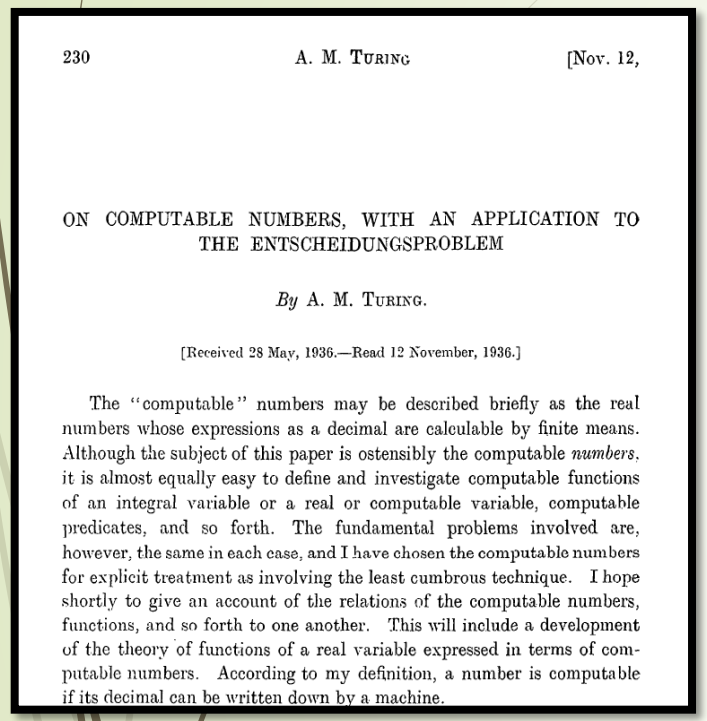


Momento en que se reconoce la invención

AF Máquina de Mealy	AF Máquina de Moore	AP	MT Máquina de Turing
1955	**1956**	**Autómata de Pila (1962–1963)**	**1936** (1954 fallece Turing) **1960:** Se introduce el concepto de autómata linealmente acotado como una restricción de la MT

MT como punto de Partida

https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf



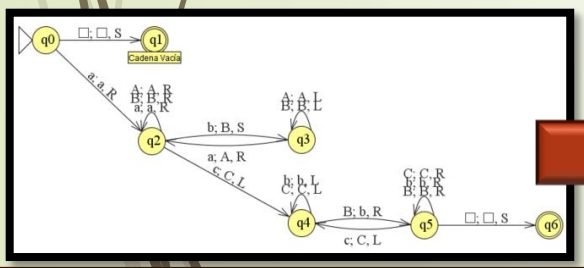
230 A. M. TURING [Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbrous technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.



- **Un modelo sencillo para definir "calcular" (La Máquina de Turing**): Turing imaginó un mecanismo básico: una cinta de papel infinita dividida en casillas, un cabezal que lee, escribe o borra símbolos, y una lista de instrucciones simples. Con este modelo abstracto demostró matemáticamente qué significa procesar información paso a paso.
- **La invención de la computadora programable (La Máquina Universal):** Demostró que no hace falta construir una máquina distinta para cada tarea. Ideó la Máquina Universal, capaz de simular a cualquier otra máquina si se le ingresan las instrucciones grabadas en la propia cinta. Este es el nacimiento del concepto de hardware general que ejecuta software.
- **Los límites de las máquinas (El Problema de la Parada):** Descubrió que hay cosas que una máquina jamás podrá resolver. Probó que es imposible diseñar un programa automático que analice a cualquier otro programa y determine con certeza si este va a terminar su trabajo o si se va a quedar colgado en un bucle infinito.
- **La respuesta al dilema matemático (Entscheidungsproblem):** Usando ese límite, resolvió un famoso desafío planteado por el matemático David Hilbert: demostró que no existe un método o algoritmo universal capaz de decidir automáticamente si cualquier afirmación matemática es verdadera o falsa.

313363117313232531117311132222322225311173111322222322222531117311132325311173111322322222631111731111323222253111731111322223222224311117311132222322222431111731113222322243111117311132222322222431111731111322232243111117311113222222322222243111173111113222222322222243111173111111322222322222531111117311111132222232222225311111173111111322322531111117311111132222232222225311111173111111336311111117

NO SE EVALUA PAPER ORIGINAL

Algoritmo de Hopcroft y Ullman

Dada una MT permite su codificación en binario



https://youtu.be/EhrDx12QTPM



Imagen generada por Copilot

Autómatas y Gramáticas



Edward Moore
Estadounidense
1925-2003



George Mealy
Estadounidense
1927-2010



Stephen Kleene
EE.UU.
1909-1994



Kurt Gödel
Austriaco
1906-1978



John Cocke
Estadounidense
1925-2002



Tadao Kasami
Japonés
n. 1930



Daniel Younger
Estadounidense



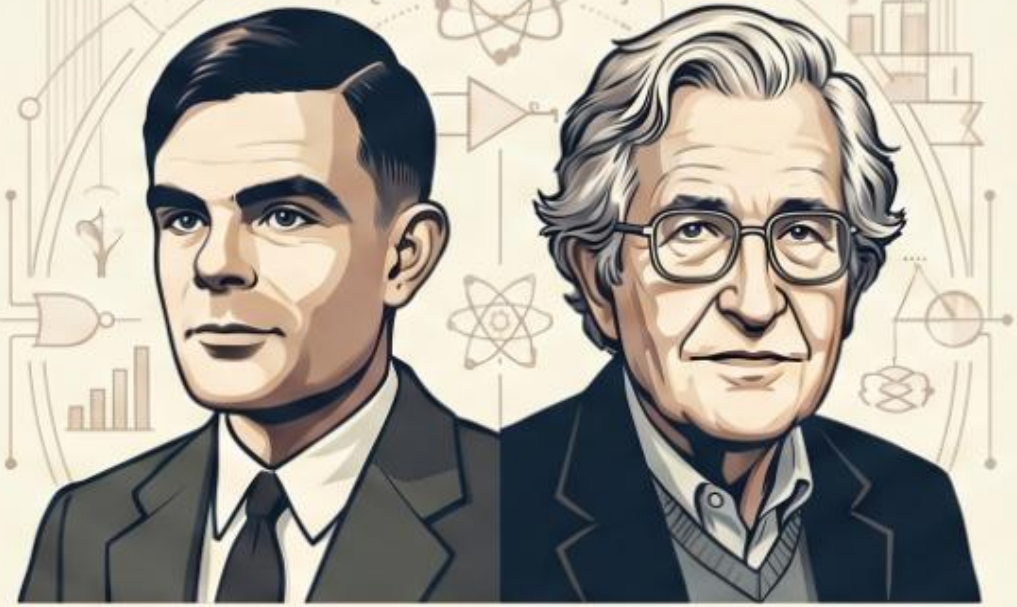
Itiroo Sakai
Japonés



Alonzo Church
EE.UU.
1903-1995



David Deutsch
Reino Unido
n. 1953



Alan Turing
(Inglés, 1912-1954)
Padre de la computación teórica y creador de la Máquina de Turing

Noam Chomsky
(Estadounidense, n. 1928)
Creador de la jerarquía que clasifica los lenguajes formales por su complejidad



Dean Arden
Estadounidense
1925-2018



Sigo-Yuki Kuroda
Japonés
1934-2009



Sheila Greibach
Estadounidense
n. 1939



George Bernard Dantzig
Estadounidense
1914-2005



Jeffrey Ullman
Estadounidense, n. 1943

John Hopcroft
Estadounidense, n. 1909



Michael Rabin
Alemán
n. 1931

Dana Scott
EE.UU.
n. 1932



John Backus
EE.UU.
1924-2007

Peter Naur
Danés
1928-2016



Robert McNaughton
Estadounidense
1924-2014



Hisao Yamada
Japonés
1930-2008



Konneth Thompson
Estadounidense
n. 1943



Janusz Brzozowski
Polaco
1935-2019

Unidad 1: Lenguajes

Unidad 2A: Lenguajes Tipo 3 – Gramáticas
Unidad 2B: Lenguajes Tipo 3 – AF

Unidad 3: Lenguajes Tipo 2

Unidad 4: Lenguajes Tipo 1 y 0

NotebookLM



Finaliza la materia

https://forms.gle/dwya3zxw6w46XjfJA